SELENIUM + TESTNG INTERVIEW MASTER GUIDE

Every commonly-asked question, explained in plain language - with memory hooks and ready-to-speak interview answers

Prepared for: jashu5446
29 September 2026

INDEX

SELENIUM + TESTNG INTERVIEW MASTER GUIDE (Fully Explained Edition) 6

SECTION 1 — "EXPLAIN YOUR FRAMEWORK" (asked in EVERY interview) 7

1.1 Understand the framework first (then the answer becomes easy) 7

1.2 The 60-second script (memorize word-for-word) 7

1.3 MEMORY HOOK for the six blocks 7

1.4 The project tree (be able to draw this) 8

1.5 Why Page Object Model? (asked immediately after the framework question) 8

SECTION 1.6 — THE END-TO-END FLOW OF ONE TEST RUN (XML -> REPORT, INCLUDING TEST DATA) 10

SECTION 2 — FRAMEWORK TYPES 12

SECTION 3 — SELENIUM CORE 13

Q: What is Selenium? 13

Q: Selenium IDE in detail — what can it actually do? 13

Q: Selenium RC in detail — what was it and WHY did it die? (classic history question) 13

Q: Selenium WebDriver in detail — the layered architecture 14

Q: Selenium Grid in detail — the component they drill the hardest 14

Q: What happens when you write driver.get()? (Architecture) 15

Q: Selenium advantages and limitations? 15

SECTION 4 — LOCATORS & XPATH (asked in every single interview) 16

Q: How many locators? Which one is best? 16

Q: Difference between CSS and XPath? 16

Q: Absolute vs Relative XPath? 16

Q: XPath functions for DYNAMIC elements (very common) 16

Q: XPath axes (the "find the neighbour" trick) 17

Q: findElement vs findElements? 17

Q: How do you find broken links? 17

SECTION 5 — WAITS (the #1 practical interview topic) 18

Q: Why do we need waits at all? 18

Q: Implicit vs Explicit vs Fluent wait? 18

Q: Why is Thread.sleep(3000) bad? 18

Q: presence vs visibility vs clickability? (a favorite trick question) 18

Q: Can we mix implicit and explicit waits? 18

SECTION 6 — WINDOWS, FRAMES, ALERTS 20

Q: close() vs quit()? 20

Q: get() vs navigate()? 20

Q: How do you handle multiple tabs/windows? 20

Q: 5 windows/tabs are open — how do you switch to the 2nd or 3rd one specifically? 20

Q: How do you handle iframes? 21

Q: How do you handle JavaScript alerts (OK/Cancel popups)? 21

Q: What about OS-level popups (like the Windows file dialog)? 21

SECTION 7 — ACTIONS, JAVASCRIPT, SCREENSHOTS, DROPDOWNS 22

Q: Actions class — what is it for? 22

Q: JavascriptExecutor — when do you actually need it? 22

Q: How do you take a screenshot? 22

Q: Handling dropdowns — two completely different cases 22

Q: Calendar / date picker? 23

Q: Auto-suggest box (like Google search suggestions)? 23

Q: File upload vs download? 23

SECTION 8 — SELENIUM 4 FEATURES (modern interviews check this) 24

SECTION 9 — TESTNG (half of every interview) 25

Q: What is TestNG? Why not JUnit? 25

Q: What does TestNG provide to Selenium? (VERY common question) 25

Q: Write the TestNG annotation hierarchy (THE most asked TestNG question) 25

Q: If no priority is given, in what order do tests run? 26

Q: What is testng.xml and why is it useful? 26

Q: Hard assert vs Soft assert (VERY common)? 26

Q: @Parameters vs @DataProvider? 26

Q: What are TestNG listeners? 27

Q: How EXACTLY do listeners take a screenshot when a test fails? (they will ask you to explain the flow) 27

Q: How do you run tests in parallel? 28

Q: What is a flaky test and how do you fix it? 28

SECTION 10 — PAGE OBJECT MODEL IN DETAIL 29

Q: Show the full pattern in code. 29

Q: PageFactory details interviewers check 29

Q: Senior-level extra: page components 29

SECTION 11 — COMMON EXCEPTIONS (memorize this table) 30

SECTION 12 — SCENARIO QUESTIONS CORNER (every scenario, fully explained) 31

Q: The element's ID changes on every page load. How do you handle it? (dynamic elements) . . . 31
Q: How do you handle CAPTCHA? . . . 31
Q: How do you handle OTP / two-factor authentication? . . . 32
Q: How do you handle Shadow DOM elements? . . . 32
Q: How do you locate SVG icons? . . . 32
Q: How do you read a specific table cell — say row 3, column 2? . . . 33
Q: How do you handle infinite scroll pages (feed keeps loading)? . . . 33
Q: The element is VISIBLE but NOT clickable. Why, and what do you do? . . . 33
Q: How do you verify a tooltip? . . . 34
Q: getAttribute() vs getText() — the difference people fumble? . . . 34
Q: isDisplayed() vs isEnabled() vs isSelected()? . . . 34
Q: How do you verify an element's color / CSS property? . . . 35
Q: How do you handle browser cookies in tests? . . . 35
Q: A test PASSES headed but FAILS headless in CI. What do you check? (the classic — with the full checklist) . 35
Q: How do you run different browsers without changing code? (browser from config) . . . 36

SECTION 13 — MAVEN + CI/CD (they drill this after the framework) . . . 38

13.1 The Maven script (memorize — this is exactly what to say) . . . 38
13.1b Q: What are the THREE functionalities of Maven? (asked directly — know each one) . . . 38
13.2 pom.xml dependencies with names + versions (know these numbers) . . . 39
13.3 Maven concepts you must be able to explain . . . 40
13.4 The CI/CD chain (say as one flow) . . . 40

SECTION 14 — JAVA ESSENTIALS ASKED ALONGSIDE SELENIUM (each with code you can speak from) . . . 41

Q: How do the OOP pillars apply to your framework? (they want Selenium mapping, not theory) . . . 41
Q: Overloading vs Overriding? (compile time vs runtime — say it with a test example) . . . 41
Q: final vs finally vs finalize? (the classic trap triple) . . . 42
Q: String vs StringBuilder — why does it matter in tests? . . . 42
Q: Which collections do you actually use in automation? (List, Set, Map — each with a REAL use) . . . 42
Q: Exception handling — checked vs unchecked, and where each appears in your framework? . . . 42
Q: == vs .equals()? (the one that separates Java people from script kiddies) . . . 43
Q: Where do you use Java 8 features in tests? (lambda + streams) . . . 43

SECTION 15 — CODING QUESTIONS THEY MAKE YOU WRITE LIVE (full code — practice until each is 5 minutes) . . . 44

15.1 Login test with POM + TestNG + DataProvider (the #1 hands-on — own it cold) . . . 44
15.2 Explicit wait until clickable, then click . . . 44
15.3 Switch between two windows and return to the parent . . . 45
15.4 Count and print all links on a page . . . 45

15.5 Select a dropdown value in all three ways . . . 45
15.6 Screenshot on failure using a listener (condensed — full flow in Section 9) . . . 45
15.7 A login form sitting inside an iframe . . . 46
15.8 Excel-driven DataProvider with Apache POI (the full pattern) . . . 46
15.9 SoftAssert all form validation messages . . . 46
15.10 IRetryAnalyzer — retry failed tests automatically . . . 47

SECTION 16 — RAPID-FIRE, FULLY EXPLAINED (final revision — every line now with its why) . . 48

SECTION 17 — CLOSING ANSWER + REVISION PLAN . . . 50

Last-day revision plan . . . 50

SECTION 18 — THE REMAINING CORNER (extras interviewers check — don't get caught here) . 51

18.1 Browser options deep-dive + SSL certificate handling (classic question) . . . 51
18.2 The THREE Selenium timeouts (a favorite trick question) . . . 51
18.3 Singleton and Factory patterns in the framework (asked in senior rounds) . . . 51
18.4 Apache POI — reading Excel (they make you WRITE this) . . . 52
18.5 Extent Reports — the actual setup code . . . 52
18.6 Cookies in detail . . . 52
18.7 API and Database testing alongside Selenium . . . 52
18.8 BDD / Cucumber (asked in a huge share of interviews) . . . 52
18.9 Appium in one breath . . . 53
18.10 QA process essentials (hybrid/manual interviews) . . . 53

SELENIUM + TESTNG INTERVIEW MASTER GUIDE (Fully Explained Edition)

Every topic below has 4 parts. Read them in this order:

1. **Understand it** — a simple explanation with a real-life example, so the concept actually makes sense.
2. **Code/Example** — what it looks like when written.
3. **MEMORY HOOK** — a short phrase to lock it in your brain.
4. **SAY THIS** — what to speak in the interview, in 20–40 seconds.

SECTION 1 — "EXPLAIN YOUR FRAMEWORK" (asked in EVERY interview)

1.1 Understand the framework first (then the answer becomes easy)

Think of a framework like a **restaurant**:

- The **waiter** (your test script) never cooks — he just takes the order to the kitchen.
- The **kitchen** (your page classes) does the actual work.
- The **recipe book** (your config/Excel files) holds the ingredients, so you never run to the market for each order.
- The **manager** (TestNG) decides which orders come first, which waiter serves which table, and checks if the food was good.
- The **bill and photos of dishes** (Extent reports + screenshots) prove what happened.

A "framework" is nothing mysterious — it is just a folder structure + rules that keep your automation **organized, reusable, and easy to fix when the application changes.**

1.2 The 60-second script (memorize word-for-word)

> *"Our framework is a **Maven-based Hybrid framework** using **Java, Selenium WebDriver, TestNG and the Page Object Model**. It has six blocks: 1. **Config block** — properties files and Excel hold the URL, browser, credentials and test data, so nothing is hardcoded in the code. 2. **Base block** — a BaseTest class that opens the browser before every test and closes it after. 3. **Pages block** — every webpage is one Java class holding its locators and actions. This is the Page Object Model. 4. **Tests block** — TestNG test classes that only call the page methods and check the results. No locators here. 5. **Utils block** — reusable helpers: waits, screenshots, Excel reader. 6. **Reports block** — Extent Reports with screenshots on failure, plus Log4j logs. The whole suite runs from a **testng.xml**, which lets me group tests and run them in parallel. The code lives in **Git**, is built by **Maven**, and runs nightly on **Jenkins**, which publishes the report and emails the team."*

1.3 MEMORY HOOK for the six blocks

"Can Big Projects Test Until Reports Come?"

Word	Block	What lives there
Can -> Config	config/	.properties + Excel (data, not code)
Big -> Base	base/	BaseTest — open/close browser
Projects -> Pages	pages/	one class per webpage
Test -> Tests	tests/	TestNG classes + assertions
Until -> Utils	utils/	waits, screenshots, Excel reader
Reports -> Reports	reports/	Extent HTML, logs, screenshots
Come -> CI	Jenkins	mvn clean test runs nightly

If you can say that one sentence, you can draw the whole framework on the whiteboard without missing a block.

1.4 The project tree (be able to draw this)

```
SeleniumTestFramework/
|-- pom.xml                    <- all JARs (dependencies) declared here
|-- testng.xml                 <- which tests to run, parallel or not
+-- src/
    |-- main/java/                          <- THE FRAMEWORK (reusable library)
    |     |-- base/BaseTest.java            -> opens/closes browser
    |     |-- pages/LoginPage.java          -> locators + login() method
    |     |-- pages/HomePage.java           -> locators + actions
    |     +-- utils/                        -> WaitUtils, ScreenshotUtils, ExcelUtils
    +-- test/
          |-- java/
          |     |-- tests/LoginTest.java            -> @Test methods + assertions
          |     +-- listeners/TestListeners.java    -> screenshot on failure
          +-- resources/
                |-- config.properties      -> url, browser, username, timeouts
                |-- testdata/ExcelData.xlsx -> data rows for DataProviders
                +-- log4j2.xml             -> logging configuration
```

Q: What is the difference between src/main/java and src/test/java? (asked when they see your tree)

Understand it: Maven forces this split on purpose —

- **src/main/java** = your **framework / library code** — Base, Pages, Utils. It gets compiled, and with `mvn install` it becomes a **JAR** that other projects can also reuse.
- **src/test/java** = the **tests that USE the framework** — your TestNG classes and listeners. It can see everything in src/main/java, but NOT the other way around.
- **src/test/resources** = test data and configs (Excel, .properties, log4j2.xml) — automatically on the test classpath, so `config.properties` and Excel paths are found at runtime.

Two conventions you should be able to talk about:

1. **Reusable-framework style (what the tree above shows):** base/pages/utils in `src/main/java`, tests in `src/test/java`. Advantage — `mvn install` builds a framework JAR that 3 different product test-suites can depend on; the framework is versioned and shared like a product.
2. **Small-project style:** everything under `src/test/java`. Totally valid for a single-suite project — many teams do it. If asked "why is everything in test?" — "the framework is used only by this one suite, so we keep it simple."

Bonus line that earns points: "Dependencies with `<scope>test</scope>` — like TestNG — are visible ONLY inside src/test/java; that's why the scope matters."

MEMORY HOOK: *"Main is the machine, Test is the test drive"* — main builds the library, test uses it.

SAY THIS: "We keep the framework — base, pages, utils — in src/main/java so it compiles as a reusable library, and the TestNG tests and listeners in src/test/java, with test data and config in src/test/resources. That way the same framework can serve multiple test suites, and test-scoped dependencies like TestNG stay isolated to the test side."

Why this structure is powerful (say this if asked "why so much structure?"):

"If the login button's ID changes tomorrow, I change it in ONE file — LoginPage.java — and all 50 tests that use login keep working. Without the framework, I would have to find and fix the ID inside 50 different test scripts."

1.5 Why Page Object Model? (asked immediately after the framework question)

Understand it: Imagine you write `driver.findElement(By.id("login")).click()` in 30 test scripts. One day the developer changes the button. Now you must edit 30 files, and you will surely miss a few. POM says: keep the locator in ONE class (the page class). Tests never touch locators directly — they only call methods like `loginPage.login(user, pass)`.

Three benefits, in plain words:

- **One place to fix** (maintainability)

- **Tests read like English** (readability): `homePage.verifyWelcomeMessage()` instead of 10 lines of locator code
- **Reuse** (the same page class serves 50 tests)

> **MEMORY HOOK:** *"Locators live with their page; tests stay clean."*

SECTION 1.6 — THE END-TO-END FLOW OF ONE TEST RUN (XML -> REPORT, INCLUDING TEST DATA)

Understand it: This is the question "walk me through the flow of your framework from start to end" — the full journey of ONE suite run, from Jenkins firing the command to the team receiving the report. Learn the 15 steps as a story:

```
Jenkins (nightly, or you running mvn clean test locally)
   |
   v  command: mvn clean test
1. MAVEN
   • reads pom.xml
   • downloads JARs (Selenium, TestNG, POI...) from Maven Central -> .m2 folder
   • surefire plugin opens testng.xml
   |
   v
2. TESTNG ENGINE reads the suite
   • registers LISTENERS (from the XML)
   • creates objects of the test classes
   |
   v
3. @BeforeSuite  -> ConfigReader loads config.properties (URL, browser, timeouts)
                    • ExtentReports object created (report session starts)
   |
   v
4. @BeforeTest   -> applies the browser parameter for this <test> block
   |
   v
5. @BeforeClass  -> WebDriverManager fetches chromedriver • browser opens
                      • page objects created: new LoginPage(driver)
   |
   v
6. DataProvider runs BEFORE the test it feeds  <-- TEST DATA ENTERS HERE
   • ExcelUtils (Apache POI) opens ExcelData.xlsx
   • reads every data row -> returns Object[][]
   • the @Test will now run ONCE PER ROW
   |
   v
7. @BeforeMethod -> fresh login, clean state for each test
   |
   v
8. @Test (row 1)
   • calls page methods: lp.login(user, pass)   <- POM layer
   • pages use WaitUtils (explicit waits)         <- Utils layer
   • Assert.assertEquals(actual, expected)         <- verdict recorded
   |
   v
9. LISTENERS fire automatically:
   • PASS -> onTestSuccess -> green in report
   • FAIL -> onTestFailure -> SCREENSHOT captured + attached to Extent, Log4j entry
   |
   v
10. @AfterMethod -> logout / cleanup
    |
    v
11. Loop back to step 7 for row 2, row 3... and every other @Test
    |
    v
12. @AfterClass -> driver.quit() (browser closes)
13. @AfterTest  -> cleanup for the <test> block
14. @AfterSuite -> extent.flush()  <- report file WRITTEN to disk
    |
    v
15. MAVEN finishes the build -> Jenkins publishes the HTML report
    -> email/Slack notification to the team
```

The 3 doors where TEST DATA enters (interviewers love this):

Door	File	What it holds	Example
1. Environment data	config.properties	same for ALL tests	URL, browser, timeouts

Door	File	What it holds	Example
2. Input data	Excel -> DataProvider	many rows, test runs once per row	username/password combos
3. Run-time params	testng.xml `<parameter>`	chosen per run	browser=chrome or firefox

SAY THIS (the 60-second spoken version):

"Jenkins triggers `mvn clean test`. Maven reads pom.xml, downloads the dependencies, and the surefire plugin loads testng.xml. TestNG registers the listeners and starts the suite. In @BeforeSuite we load config.properties and start the Extent report. @BeforeClass opens the browser and creates the page objects. The DataProvider then reads test data rows from Excel using Apache POI, so each test runs once per row. Inside the @Test we call page object methods with explicit waits and assert the results — if anything fails, the listener's onTestFailure captures a screenshot and attaches it to the report automatically. After all tests, @AfterClass quits the driver and @AfterSuite flushes the Extent report, which Jenkins publishes and emails to the team."

MEMORY HOOK (one line — memorize this):

"Jenkins fires Maven, Maven loads TestNG, Config feeds Base, Excel feeds Data, Pages serve Tests, Listeners catch failures, AfterSuite writes the report, Jenkins emails it."

Short version of the same hook: **Build -> Suite -> Base -> Data -> Test -> Listener -> Report -> Email.**

SECTION 2 — FRAMEWORK TYPES

Understand each with one line:

Type	What it means in simple words	Example	
Linear	One long recorded script, top to bottom. No functions.	Record & playback	
Modular	Break the app into small functions/modules and reuse them.	`login()`, `search()` used everywhere	
Data-driven	ONE test, MANY rows of data — run the same steps with different inputs.	Login test runs 5 times with 5 usernames from Excel	
Keyword-driven	The test steps themselves are written in Excel as keywords (Click, Enter, Verify) and code reads and executes them.	Row: `Click	loginBtn`
Hybrid	A mix — usually POM + data-driven + modular.	Most real company frameworks	

SAY THIS: "Ours is a Hybrid framework — POM for structure, plus data-driven testing where TestNG DataProviders read rows from Excel using Apache POI."

MEMORY HOOK: *"L-M-D-K-H: Lazy Monkeys Don't Keep Hybrids"* — Linear, Modular, Data-driven, Keyword, Hybrid.

SECTION 3 — SELENIUM CORE

Q: What is Selenium?

Understand it: Selenium is a free, open-source set of tools that lets a program **control a web browser like a human** — open pages, click buttons, type text, read text. It cannot think; it only does what your code tells it.

Four components:

1. **Selenium IDE** — a browser plugin that records your clicks and replays them. Good only for quick demos. No logic, no loops, no assertions.
2. **Selenium RC** — the OLD, retired tool (Remote Control). It worked by injecting JavaScript into the page. Know it only for history questions.
3. **Selenium WebDriver** — what everyone actually uses today. Your Java code sends commands to a small driver program (chromedriver.exe) which moves the real browser.
4. **Selenium Grid** — run the same tests on many machines/browsers at the same time, in parallel.

MEMORY HOOK: *"I-R-W-G: Ide, RC, WebDriver, Grid — only W works for real work."*

The Selenium Suite at a glance (say this table from memory)

Component	One-line identity	Status today	Use it for
Selenium IDE	Record & playback browser plugin	Active	quick demos, locator discovery
Selenium RC	JavaScript-injection server	**Retired** (since Selenium 3)	history questions only
Selenium WebDriver	W3C protocol -> driver -> browser	**THE core everyone uses**	all real automation
Selenium Grid	Parallel runs across machines	Active (Grid 4)	CI suites at scale

Q: Selenium IDE in detail — what can it actually do?

Understand it: IDE is a **Chrome/Firefox plugin** with record & playback. You click around, it records every action as a command (`click id=login`), and you can replay it later.

What most candidates DON'T know (so say it and stand out):

- Tests are saved as `.side` **files** (a project format), and can be run from the command line with `selenium-side-runner` — so IDE tests can even run in CI.
- The new IDE has **control flow** — `if / else / while / times` commands, and you can export a recorded test as **real code** (Java, Python, JavaScript, JUnit/TestNG) — the classic use: record a flow, export it, and clean it up into your POM framework.
- **Limits (why no company relies on it):** no proper assertions, no framework (no POM, no grouping, no reports), breaks easily when the UI changes, no parallel execution, and recorded locators are often ugly (absolute XPath).

SAY THIS: "We use IDE occasionally to discover flows or export a skeleton, then move everything into the WebDriver + TestNG framework — IDE is a sketchbook, not the factory."

MEMORY HOOK: *"IDE = record, replay, regret."*

Q: Selenium RC in detail — what was it and WHY did it die? (classic history question)

Understand it: RC = **Remote Control**, the tool from the mid-2000s, before WebDriver existed.

How RC worked (its architecture — 3 steps):

1. You start the **selenium-server** (a Java server you run separately).
2. Your test code sends **HTTP commands** to that server ("click login", "type user").
3. The server **injects JavaScript (called Selenium Core) into the browser page**, and that JavaScript drives the page — clicks, typing, everything.

So RC controlled the browser **from INSIDE the page using injected JavaScript** — that one design decision caused every one of its problems:

Why WebDriver replaced RC (the 4 reasons):

1. **Same-Origin Policy** — browsers block JavaScript on a page from talking to a different domain, so RC needed an awkward proxy setup for cross-domain testing. This is THE classic answer.
2. **JavaScript sandbox limits** — injected JS couldn't do things a real user does with the OS (native dialogs, file dialogs, real mouse movement outside the page).
3. **Slower and less stable** — every command went through the server -> injection -> JS execution chain.
4. **Not browser-native** — WebDriver instead ships a real **driver binary per browser** that talks to the browser natively at the C++ level.

History dates that impress: RC + WebDriver merged into **Selenium 2 (2011)**; RC moved to a legacy `selenium-rc` package in **Selenium 3 (2016)**; the WebDriver API became a **W3C standard (2018)** and Selenium 4 is fully W3C.

MEMORY HOOK: *"RC drove the browser with JavaScript inside the page — and died of the Same-Origin disease."*

Q: Selenium WebDriver in detail — the layered architecture

Understand it: "Selenium WebDriver" is actually a stack of layers — know all four:

1. **Language bindings** — the client libraries (Java, Python, C#...) — the API you write against.
2. **The W3C WebDriver protocol** — the standardized command language ("open page", "click element") sent as HTTP JSON.
3. **Browser drivers** — chromedriver, geckodriver, msedgedriver — one small binary per browser that RECEIVES the commands and drives the real browser.
4. **The real browsers** — Chrome, Firefox, Edge.

Two extra facts that earn points:

- **JSON Wire vs W3C:** before Selenium 4 the protocol was the old "JSON Wire Protocol" (non-standard, browser quirks); Selenium 4 speaks **W3C** — same standard every browser implements, fewer compatibility bugs.
- **Stateless:** every command is a fresh HTTP request — the driver holds no open connection to your code. (Compare: Playwright keeps ONE WebSocket pipe — see the Playwright guide!)

MEMORY HOOK: *"Bindings speak W3C over HTTP to the driver, the driver drives the browser."*

Q: Selenium Grid in detail — the component they drill the hardest

Understand it: Your suite takes 3 hours on one laptop. Grid lets the SAME tests run **in parallel on many machines and browsers at the same time** — 40 minutes on 5 nodes. It also gives cross-browser/OS coverage (one Grid, Chrome on Linux + Safari on Mac).

Grid 3 (classic) — the HUB and NODE story everyone asks

- **Hub** — the central receptionist. It receives ALL test requests from your code and assigns each to an available node. You don't run a browser on the hub.
- **Nodes** — the worker machines. Each node registers with the hub and advertises what it has: "I have Chrome 130 on Windows, Firefox on Linux", with N browser slots.
- **Starting them (know the commands):**

```
java -jar selenium-server-standalone.jar -role hub
java -jar selenium-server-standalone.jar -role node -hub http://<hub-ip>:4444
```

- **Pointing your test at the Grid:** instead of `new ChromeDriver()`, you use **RemoteWebDriver** with the hub URL + capabilities:

```
ChromeOptions options = new ChromeOptions();
WebDriver driver = new RemoteWebDriver(
```

```
        new URL("http://hub-ip:4444"), options);
// same WebDriver API — the test doesn't know or care where the browser actually runs
```

That's the elegant point: **your test code stays identical** — only the driver creation changes.

Grid 4 (current) — new architecture, three modes

Grid 4 was rebuilt. Inside it, the old hub splits into components: **Router** (entry point), **Distributor** (assigns tests to nodes), **Session Map** (remembers which node runs which session), **Event Bus** (their internal messaging). You don't need the internals memorized — know the THREE ways to run it:

1. **Standalone** — one command, everything in one JVM, on one machine:

`java -jar selenium-server.jar standalone`

2. **Hub and Node** — the classic two-machine setup, now:

`java -jar selenium-server.jar hub` + `java -jar selenium-server.jar node`

3. **Fully Distributed** — each component (router, distributor, sessions, event bus, nodes) runs as its own process on its own machine — for big farms.

Plus: Grid 4 is fully W3C, has a built-in **UI dashboard** (http://machine:4444), supports **Docker** (nodes as containers), live node monitoring, and external Redis for the session map.

Grid vs simply running in Jenkins? Say this: "Grid parallelizes BROWSERS across machines with session routing; Jenkins parallelizes JOBS. We combine them — Jenkins runs testng.xml sharded into Grid."

MEMORY HOOK (Grid 3): *"One receptionist (hub), many workers (nodes)."*

MEMORY HOOK (Grid 4 modes): *"Standalone = all alone, Hub+Node = classic pair, Distributed = every part gets its own machine."*

Q: What happens when you write driver.get()? (Architecture)

Understand it step by step:

1. Your Java code (`driver.get("...")`) does NOT touch the browser directly.
2. It converts your command into an **HTTP request** (like a small web message) and sends it to **chromedriver.exe** (or geckodriver for Firefox).
3. The driver receives the request and controls the real Chrome/Firefox browser.
4. The browser does the action and sends back a response.

So the chain is: **Your code -> HTTP -> driver exe -> real browser.**

In Selenium 4 this message format is the **W3C WebDriver protocol** — a worldwide standard, which makes commands behave the same in every browser.

MEMORY HOOK: *"Code -> Driver -> Browser, always through HTTP."*

Q: Selenium advantages and limitations?

Advantages (say 4): free and open-source; works with Java/Python/C# and all major browsers; big community; fits perfectly with TestNG/Maven/Jenkins.

Limitations (know these — interviewers check honesty):

- Web browsers only — cannot automate desktop apps (that's WinAppDriver) or mobile apps (that's Appium).
- Cannot read/handle CAPTCHA or OTP by design — these exist to stop bots.
- No built-in reports or test management — that's why we add TestNG, Extent, etc.

MEMORY HOOK: *"Web only, no captcha, brings no reports of its own."*

SECTION 4 — LOCATORS & XPATH (asked in every single interview)

Q: How many locators? Which one is best?

Understand it: To click or type anywhere, Selenium first needs an "address" of that element in the HTML. There are 8 ways to give that address:

`id`, `name`, `className`, `tagName`, `linkText`, `partialLinkText`, `cssSelector`, `xpath`

Which is best and WHY (this is what interviewers want):

- **id** is best because a good developer makes it unique on the page — fast and exact, like an Aadhaar number.
- **name** is second — usually unique too.
- **CSS** — good for combinations of tag + attribute.
- **XPath** — most powerful (can search text, go up to parents) but slowest — the last resort.

Order to say: id -> name -> className -> CSS -> XPath.

> **MEMORY HOOK:** *"ID is the King; XPath is the last resort."*

Q: Difference between CSS and XPath?

Understand with an example. Suppose a button:

```
<button id="submit" class="btn">Save</button>
```

- CSS: `button#submit` or `button.btn`
- XPath: `//button[@id='submit']`

The big practical difference: **XPath can search UPWARD (find a parent), CSS cannot.** XPath can also match visible text (`//button[text()='Save']`) — CSS cannot match text at all.

If you need "the row that contains the word Laptop, then its price cell" — you need XPath axes.

> **MEMORY HOOK:** *"CSS goes down only; XPath goes up and down."*

Q: Absolute vs Relative XPath?

Absolute: starts with a single slash `/`, gives the FULL path from the root:

`/html/body/div[2]/div[1]/form/input[1]`

Problem: if ANY tag in the chain moves, the whole path breaks. Like giving your address as "Earth -> India -> 3rd building -> 2nd floor -> left door".

Relative: starts with double slash `//`, searches anywhere in the page:

`//input[@id='username']`

This is what you should always use.

> **MEMORY HOOK:** *"One slash = fragile; two slashes = flexible."*

Q: XPath functions for DYNAMIC elements (very common)

The problem: some sites generate ids like `username_83721` — the number changes every page load, so `@id='username_83721'` fails next time.

The fix — three tools:

```
// contains() — matches any id that CONTAINS the word 'username'
//input[contains(@id,'username')]

// starts-with() — matches ids that BEGIN with 'username_'
//input[starts-with(@id,'username_')]

// text() — match by visible text
```

```
//button[text()='Login']
//a[contains(text(),'Forgot Password')]
```

MEMORY HOOK: *"ID keeps changing? contains() it."*

Q: XPath axes (the "find the neighbour" trick)

Understand it: Axes mean "from this element, now look at its relatives."

Classic interview table question — "How do you find the price of a product when you only know its name?"

```
<tr>
  <td>Laptop</td>      <!-- name -->
  <td>45000</td>       <!-- price -->
</tr>
```

```
// find the cell containing 'Laptop', then the NEXT cell (sibling) in the same row
//td[text()='Laptop']//following-sibling::td[1]
```

Other axes: `parent`, `ancestor`, `child`, `preceding-sibling`, `following`, `preceding`.

MEMORY HOOK: *"following = BELOW in the HTML, preceding = ABOVE."*

Q: findElement vs findElements?

Understand it:

- `findElement` -> "give me ONE element" -> returns the FIRST match. If nothing matches -> **throws NoSuchElementException** (test crashes).
- `findElements` -> "give me ALL matching elements" -> returns a List. If nothing matches -> **returns an EMPTY list, no error** — like searching your contacts for "Xyz": one search shows an error popup, the other just shows an empty list.

Classic use — count all links on a page:

```
List<WebElement> links = driver.findElements(By.tagName("a"));
System.out.println(links.size());
```

MEMORY HOOK: *"One throws, the List lies low (comes back empty)."*

Q: How do you find broken links?

Understand it: Selenium cannot open a URL and check if it is alive — but plain Java can. So:

1. Collect all `<a>` tags (findElements).
2. Read each one's `href` attribute (`getAttribute("href")`).
3. Skip empty and javascript links.
4. Send each URL an HTTP request in Java (HttpURLConnection) and read the **status code**: 200 = fine, 404/400 = broken.
5. Print/report all broken ones.

MEMORY HOOK: *"Status 200 = alive; 4xx = dead."*

SECTION 5 — WAITS (the #1 practical interview topic)

Q: Why do we need waits at all?

Understand it: Selenium runs at computer speed; the page loads at network speed. You ask Selenium to click a button that hasn't appeared yet -> crash. Waits = telling Selenium "wait for the page to be ready, not just run blindly." This is called **synchronization**.

Q: Implicit vs Explicit vs Fluent wait?

Implicit wait — the "global blanket":

```
driver.manage().timeouts().implicitlyWait(10, TimeUnit.SECONDS);
```

You set it once, and it applies to EVERY findElement in the whole script: "if you don't find the element, keep trying for up to 10 seconds before giving up."

Simple, but dumb — it can only wait for the element to EXIST. It cannot wait for a button to become clickable or a loader to disappear.

Explicit wait — the "targeted sniper":

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
WebElement btn = wait.until(
    ExpectedConditions.elementToBeClickable(By.id("submit")));
btn.click();
```

You wait for ONE specific condition on ONE specific element: clickable, visible, text present, alert shown, etc. This is what a professional framework mostly uses.

Fluent wait — "a sniper with a custom clock":

```
Wait<WebDriver> wait = new FluentWait<>(driver)
    .withTimeout(Duration.ofSeconds(30))      // max total wait
    .pollingEvery(Duration.ofSeconds(3))      // check every 3 seconds
    .ignoring(NoSuchElementException.class);   // don't crash while polling
```

Same idea as explicit, but you control HOW OFTEN to check and WHICH errors to ignore while checking. Useful for slow-loading elements that throw StaleElement errors.

MEMORY HOOK: *"Implicit = blanket over everything; Explicit = one element, one condition; Fluent = explicit with a timer and patience."*

Q: Why is Thread.sleep(3000) bad?

Understand it: Thread.sleep **always sleeps the full 3 seconds**, even if the page was ready after 200 milliseconds — so the test is unnecessarily slow. And if the page needs 5 seconds? It still crashes. Waits are smart: they keep checking and continue the MOMENT the condition is true.

MEMORY HOOK: *"Sleep is blind waiting; waits are smart waiting."*

Q: presence vs visibility vs clickability? (a favorite trick question)

- **presence** — the element exists in the HTML (the DOM), but might be invisible (hidden).
- **visibility** — it exists AND is actually rendered on screen (you could see it).
- **clickable** — visible AND enabled (a disabled greyed-out button is visible but NOT clickable).

MEMORY HOOK: *"In DOM -> on screen -> ready to use."*

Q: Can we mix implicit and explicit waits?

Best answer: **No.** When both are set, each polling cycle of the explicit wait ALSO waits the implicit time — the delays stack up and become unpredictable. Selenium's own docs advise against mixing.

MEMORY HOOK: *"Never mix blanket and sniper."*

SECTION 6 — WINDOWS, FRAMES, ALERTS

Q: close() vs quit()?

close() — closes ONLY the current browser window. The session (the connection between your code and the driver) is still alive; other windows stay open.

quit() — closes ALL windows AND ends the session completely. If you forget quit(), chromedriver.exe processes stay running in the machine's memory like zombies.

MEMORY HOOK: *"Close one, Quit ALL + kill the session."*

Q: get() vs navigate()?

`driver.get("url")` — open a page and wait for it to load.

`driver.navigate().to("url")` — opens too, but navigate() also gives you **back(), forward(), refresh()** — the browser's back/forward arrows in code form.

MEMORY HOOK: *"get goes; navigate goes AND can come back."*

Q: How do you handle multiple tabs/windows?

Understand it: Every browser window/tab has an ID (handle). Selenium can only work in ONE window at a time — you must "switch" your focus by giving the ID.

```
String parent = driver.getWindowHandle();     // THIS window's ID (single String)

driver.findElement(By.id("openNew")).click();  // new tab opens

Set<String> all = driver.getWindowHandles();   // ALL window IDs (a Set)

for (String handle : all) {
    if (!handle.equals(parent)) {
        driver.switchTo().window(handle);       // move focus to the new tab
    }
}
// do your work in the new tab...
driver.close();                                // close the tab
driver.switchTo().window(parent);               // come back home
```

MEMORY HOOK: *"Handle = THIS one (String); HandleS = ALL of them (Set). The 'S' is everything."*

Q: 5 windows/tabs are open — how do you switch to the 2nd or 3rd one specifically?

Understand it: the driver drives exactly ONE window at a time — the "active" one — and every command goes there until you switch. `getWindowHandles()` returns a **Set**, and a Set has no guaranteed order: "the 3rd handle in the Set" is not promised to be the 3rd window you opened, and it can even shuffle between runs. That's why blind indexing is the junior trap, and the two professional answers are: index the LIST when you know the state is stable, or switch-and-match by title/URL — the one interviewers actually want to hear (and the one you wrap into a reusable utility method).

Answer 1 — by index (fast, when order is known): Set -> List, then get(n)

```
Set<String> all = driver.getWindowHandles();    // all open windows (a Set)
List<String> tabs = new ArrayList<>(all);        // Set -> List: now indexable

driver.switchTo().window(tabs.get(0));           // 1st window (usually the original)
driver.switchTo().window(tabs.get(1));           // 2nd window
driver.switchTo().window(tabs.get(2));           // 3rd window — IF order is stable
```

Answer 2 — by title or URL (the stable, reusable way):

```
// the utility method interviewers make you write on the spot
public static void switchToWindowByTitle(WebDriver driver, String title) {
    for (String handle : driver.getWindowHandles()) {
        driver.switchTo().window(handle);                // peek inside
        if (driver.getTitle().contains(title)) return;    // found it — stay here
```

```
    }
    throw new RuntimeException("No window with title: " + title);
}

switchToWindowByTitle(driver, "Payment");   // lands on the payment tab, wherever it sits
```

Why the loop is safe: switching to the handle you are already on is a no-op — so peeking into every window costs nothing and cannot break state.

The version trap to recite: `tabs.get(0)` is *usually* the original window because drivers *tend* to keep insertion order — but it is NOT in the contract. Title/URL matching is the only approach guaranteed correct, which is exactly why it belongs in the shared framework code, not copy-pasted per test.

Coming back home: save the parent handle BEFORE any new window opens — `String parent = driver.getWindowHandle();` — then `driver.switchTo().window(parent)` returns you no matter how many tabs opened in between.

MEMORY HOOK: *"Set has no order — index is luck, title/URL is truth. Save the parent first; peeking costs nothing."*

SAY THIS: "getWindowHandles returns a Set, and Sets carry no order guarantee, so blind indexing is fragile. If I just opened the tabs and know the state, I convert the Set to a List and switch by index. The stable way — and what I keep in a shared utility — is looping the handles, switching into each, and matching by title or URL, because switching to the current window is a no-op so the scan is free. And I always save the parent handle before opening anything, so I can always come home."

Q: How do you handle iframes?

Understand it: An iframe is a page INSIDE a page (like a TV inside a wall). Selenium can only see the MAIN page. If your element is inside the iframe, Selenium is "standing outside the room" and cannot touch it — until you **switch into** the frame.

```
driver.switchTo().frame(0);              // by index (0 = first frame)
driver.switchTo().frame("frameName");    // by name or id
driver.switchTo().frame(element);        // by the frame WebElement
driver.switchTo().defaultContent();      // step back out to the main page
```

How do you KNOW an element is in a frame? In DevTools, the element sits under a `<iframe>` tag.

MEMORY HOOK: *"Frames are rooms: switchTo to enter, defaultContent to come home."*

Q: How do you handle JavaScript alerts (OK/Cancel popups)?

Understand it: These are the browser's own grey popups (alert/confirm/prompt). They are NOT part of the HTML — so locators NEVER work on them. You must switch to them first:

```
Alert alert = driver.switchTo().alert();
alert.getText();       // read the message
alert.accept();        // click OK
alert.dismiss();       // click Cancel
alert.sendKeys("hi");  // type into a prompt popup
```

MEMORY HOOK: *"Accept = OK, Dismiss = Cancel — and no locator ever works on alerts."*

Q: What about OS-level popups (like the Windows file dialog)?

That dialog belongs to the operating system, not the browser — Selenium cannot touch it. Use Java's **Robot** class (simulates real keyboard/mouse) or an AutoIt script.

BUT say this first: "For file upload, most sites use `<input type="file">` — then I don't open any dialog at all; I just `element.sendKeys("C:/path/file.png")`. Robot is only the plan B."

MEMORY HOOK: *"Input tag? sendKeys. Real OS dialog? Robot."*

SECTION 7 — ACTIONS, JAVASCRIPT, SCREENSHOTS, DROPDOWNS

Q: Actions class — what is it for?

Understand it: Normal `.click()` and `.sendKeys()` cover simple cases. The Actions class handles **mouse gestures**: hover, right-click, double-click, drag and drop.

```
Actions a = new Actions(driver);
a.moveToElement(menu).perform();            // hover — makes hidden submenus appear
a.contextClick(element).perform();          // right-click
a.doubleClick(element).perform();          // double-click
a.dragAndDrop(source, target).perform();   // drag and drop
a.keyDown(Keys.CONTROL).click(item).keyUp(Keys.CONTROL).perform(); // multi-select
```

Two things people forget in interviews: (1) every chain must end with `.perform()` or nothing happens; (2) keyboard combos (Ctrl, Shift) also come from Actions + the Keys enum.

MEMORY HOOK: *"Mouse = Actions, and without perform() it's only a plan."*

Q: JavascriptExecutor — when do you actually need it?

Understand it: Sometimes the normal click fails because an ad, spinner, or invisible overlay covers the button ("ElementClickInterceptedException"). Or you need to scroll. Then you ask the browser to run raw JavaScript directly:

```
JavascriptExecutor js = (JavascriptExecutor) driver;
js.executeScript("window.scrollBy(0, 500)");                // scroll down 500px
js.executeScript("arguments[0].click();", element);          // force a click
js.executeScript("window.scrollTo(0, document.body.scrollHeight)"); // scroll to page end
```

Say this too (shows maturity): "I use JS as the LAST option, because a JS click bypasses the real user events — if it passes, it doesn't prove a real user could click it."

MEMORY HOOK: *"When Selenium clicks fail, JS never fails — but keep it as plan B."*

Q: How do you take a screenshot?

```
File src = ((TakesScreenshot) driver).getScreenshotAs(OutputType.FILE);
FileUtils.copyFile(src, new File("screenshots/loginFail.png"));
```

The cast `(TakesScreenshot) driver` works because the driver class implements that interface. (Full automatic-on-failure flow is in Section 9 with listeners.)

MEMORY HOOK: *"Cast, capture, copy — 3 steps."*

Q: Handling dropdowns — two completely different cases

Case 1 — real `<select>` tag -> use the Select class:

```
Select s = new Select(driver.findElement(By.id("country")));
s.selectByVisibleText("India");   // by shown text
s.selectByValue("in");             // by the value attribute
s.selectByIndex(2);                // by position, 0-based
s.getOptions();                    // all options — count/verify them
s.getFirstSelectedOption();         // what is currently chosen
```

Case 2 — "fake" dropdown (div/ul/li — Bootstrap, React) -> Select class does NOT work:

click to open it, then find all options with findElements and loop:

```
driver.findElement(By.id("dropdownToggle")).click();
List<WebElement> options = driver.findElements(By.cssSelector("ul.dropdown li"));
for (WebElement o : options) {
    if (o.getText().equals("India")) { o.click(); break; }
}
```

MEMORY HOOK: *"Select tag -> Select class; fake dropdown -> click and loop."*

Q: Calendar / date picker?

Two honest answers:

1. If the input field accepts typing — just `sendKeys("24/09/2026")`. Done.
2. If typing is blocked — navigate: click the "next month" arrow until the right month shows, then click the day cell: `//td[text()='24']`.

MEMORY HOOK: *"Type if allowed, else travel month by month."*

Q: Auto-suggest box (like Google search suggestions)?

```
driver.findElement(By.name("q")).sendKeys("selenium");
List<WebElement> suggestions =
    driver.findElements(By.cssSelector("ul li div"));   // wait for them first!
for (WebElement s : suggestions) {
    if (s.getText().contains("selenium webdriver")) { s.click(); break; }
}
```

Pattern: **type -> wait -> collect all suggestions -> loop -> click the match.**

MEMORY HOOK: *"Type, wait, find all, loop, click."*

Q: File upload vs download?

Upload: if the site uses `<input type="file">` — `element.sendKeys("path/to/file.pdf")`. No dialog, no Robot.

Download: tell Chrome WHERE to save (browser options with `download.default_directory`), trigger the download, then verify with Java: `new File(".../report.pdf").exists()` inside a small wait loop.

MEMORY HOOK: *"Upload = sendKeys; Download = set the folder + File.exists()."*

SECTION 8 — SELENIUM 4 FEATURES (modern interviews check this)

Understand it: Selenium 4 (2021 onwards) modernized everything around the W3C standard. Five things to know:

1. **W3C WebDriver protocol** — commands are now a worldwide standard, so the same code behaves the same on Chrome/Firefox/Edge. (Before, each browser spoke a slightly different "dialect" — JSON Wire Protocol — causing weird browser-specific bugs.)
2. **Relative locators** — find an element by POSITION instead of attributes: "the input just BELOW the Email label":

```
driver.findElement(RelativeLocator.with(By.tagName("input")).below(emailLabel));
```

Options: `above(), below(), toLeftOf(), toRightOf(), near()`.

3. **New Grid** — Grid 4 is easier to run and works with Docker. (Grid 3 = hub + node; Grid 4 = router + distributor + sessions + event bus — just say "distributed architecture".)
4. **Chrome DevTools (CDP) support** — simulate slow network / offline, fake geolocation, read browser console logs — great for testing "what if the user's internet is bad".
5. **New window/tab + Shadow DOM support** — `driver.switchTo().newWindow(WindowType.TAB)`, and `element.getShadowRoot()` to reach elements inside shadow DOM (which XPath can't otherwise touch).

MEMORY HOOK: *"W-R-N-C-S: Why Really New Cool Stuff"* -> W3C, Relative locators, New Grid, CDP, Shadow DOM.

SECTION 9 — TESTNG (half of every interview)

Q: What is TestNG? Why not JUnit?

Understand it: TestNG = "Test Next Generation". It is a framework that **manages your tests** — which run first, what data they use, what happens before/after each test, what counts as pass/fail, and what report comes out.

Why it beats old JUnit in one line: JUnit ran tests; TestNG **orchestrates** them — grouping, ordering, dependencies, parallel runs, data files, retries.

MEMORY HOOK: *"NG = Next Generation — the generation after JUnit."*

Q: What does TestNG provide to Selenium? (VERY common question)

Understand it: Selenium by itself is only a **remote control for the browser** — it can open, click, type, read. It has NO idea what a "test" is. TestNG is the brain on top:

Selenium alone cannot...	TestNG gives you
mark "this method is a test"	`@Test` annotation
do setup before / cleanup after	`@BeforeMethod`, `@AfterMethod`, `@BeforeSuite`...
check expected vs actual	`Assert`, `SoftAssert`
control order/dependencies	`priority`, `dependsOnMethods`, `groups`
run in parallel	`parallel` + `thread-count`
feed data rows	`@DataProvider`, `@Parameters`
auto-retry flaky tests	`IRetryAnalyzer`
produce a report	built-in HTML + Extent via listeners
run selected combinations	`testng.xml`

MEMORY HOOK: *"Selenium DRIVES, TestNG DECIDES."* (drives the browser vs decides what/when/how and judges pass-fail.)

SAY THIS: "Selenium only automates browser actions — it cannot organize, assert, or report. TestNG adds the test layer: @Test annotations, before/after hooks, assertions, grouping, priorities, DataProviders, parallel execution, listeners and retry logic."

Q: Write the TestNG annotation hierarchy (THE most asked TestNG question)

```
@BeforeSuite -> @BeforeTest -> @BeforeClass -> @BeforeMethod -> @Test -> @AfterMethod -> @AfterClass ->
```

Understand it with the hotel analogy:

- **Suite** = the whole hotel (all testng.xml suites) — biggest scope
- **Test** = one floor (a `<test>` block in the XML)
- **Class** = one room (a test class)
- **Method** = the bed — the smallest, most personal space (one @Test method)

Setup runs **biggest -> smallest** (open the hotel before you make the bed).

Teardown runs **smallest -> biggest** (leave the bed before locking the hotel).

Practical mapping interviewers like:

- `@BeforeSuite` — start the Extent report, load config.properties (once for everything)
- `@BeforeTest` — set the browser for this `<test>` block
- `@BeforeClass` — open browser once for the whole class
- `@BeforeMethod` — fresh login before EVERY test (clean state each time)

- `@AfterMethod` — logout / screenshot on failure
- `@AfterSuite` — flush (write) the report

MEMORY HOOK: *"Biggest->smallest for setup, smallest->biggest for teardown."*

Q: If no priority is given, in what order do tests run?

Alphabetically by method name (a, b, c...). Fix it with:

- `@Test(priority = 1)` — lowest number runs first; negatives allowed; default 0
- `@Test(dependsOnMethods = "createOrder")` — run only after that method PASSES; if it fails, this test is SKIPPED (shows as skipped, not failed — good talking point)
- `alwaysRun = true` — run even if the depended-on method failed
- `enabled = false` — switch a test off without deleting it
- `invocationCount = 5` — run the same test 5 times (for stability checks)
- `timeOut = 5000` — fail the test if it exceeds 5 seconds
- `expectedExceptions = ArithmeticException.class` — pass only if that exception occurs

MEMORY HOOK: *"No priority = A before B before C."*

Q: What is testng.xml and why is it useful?

Understand it: testng.xml is a control panel for your suite — you choose which tests, which groups, which browser, how many threads — WITHOUT touching Java code. A manual tester can also edit it.

```
<suite name="Regression" parallel="tests" thread-count="3">
  <listeners>
    <listener class-name="listeners.TestListeners"/>
  </listeners>
  <test name="Smoke">
    <parameter name="browser" value="chrome"/>
    <groups><run><include name="smoke"/></run></groups>
    <classes><class name="tests.LoginTest"/></classes>
  </test>
</suite>
```

MEMORY HOOK: *"testng.xml is the remote control of the suite."*

Q: Hard assert vs Soft assert (VERY common)?

Understand it:

- **Hard assert (`Assert`)** — a strict examiner: first wrong answer -> stop the test immediately. Later checks never run, so you see only ONE error.
- **Soft assert (`SoftAssert`)** — a patient examiner: notes every wrong answer and lets the test continue; at the end, `assertAll()` reports ALL failures together.

```
SoftAssert sa = new SoftAssert();
sa.assertEquals(actualTitle, "Dashboard");     // even if this fails...
sa.assertTrue(welcomeMsg.contains("Welcome")); // ...this still runs
sa.assertAll();   // <- without this line, soft assert failures are silently ignored!
```

When to use which: verifying 10 form-field error messages -> SoftAssert (want all 10); verifying login landed on Dashboard before acting -> hard Assert (no point continuing on the wrong page).

MEMORY HOOK: *"Hard = halt. Soft = collect all, spill at assertAll."*

Q: @Parameters vs @DataProvider?

- **@Parameters** — takes a value FROM testng.xml (`<parameter name="browser" value="chrome"/>`). ONE value per run. Perfect for environment/browser selection.

```
@Test @Parameters("browser")
public void test(String browser) { ... }
```

- **@DataProvider** — a method in Java that returns `Object[][]` (a table of rows). The test runs ONCE PER ROW. Perfect for many data sets.

```
@DataProvider(name = "loginData")
public Object[][] getData() {
    return new Object[][] {
        {"admin", "admin123", true},
        {"admin", "wrongpw",  false}
    };
}
@Test(dataProvider = "loginData")
public void loginTest(String user, String pass, boolean expected) { ... }
```

2 rows = 2 test runs. In real frameworks the Object[][] is usually filled by reading Excel with Apache POI.

- **@Factory** — creates multiple instances of the whole test class with different data. (Just know the name and one line.)

MEMORY HOOK: *"Parameter = one value from XML; DataProvider = many rows, one run each."*

Q: What are TestNG listeners?

Understand it: A listener is like a **security camera in the exam hall** — it doesn't take the exam, it just WATCHES every event (test started, passed, failed, skipped) and reacts. Your test code doesn't need to know it's being watched.

The important ones:

- **ITestListener** — onTestStart / onTestSuccess / **onTestFailure** / onTestSkipped -> here we take screenshots and write to the report
- **ITestResult** — the object carrying which test, what status, what exception
- **IRetryAnalyzer** — retry a failed test automatically (flaky test protection): return `true` from `retry()` to run it again, up to a max count
- **IAnnotationTransformer, ISuiteListener, IExecutionListener** — advanced; just recognize the names

Q: How EXACTLY do listeners take a screenshot when a test fails? (they will ask you to explain the flow)

The 4-step flow:

1. The **BaseTest** class keeps the driver in a `public static` field so any class can reach it.
2. A class `TestListeners` implements **ITestListener** and overrides `onTestFailure(ITestResult result)`.
3. When any test fails, TestNG automatically calls that method. Inside it: take the driver -> cast to `TakesScreenshot` -> capture -> save with the test's name.
4. Register the listener once (in testng.xml) -> **every** failing test gets a screenshot automatically, and the tests themselves contain ZERO screenshot code.

```
public class BaseTest {
    public static WebDriver driver;

    @BeforeMethod
    public void setUp() {
        WebDriverManager.chromedriver().setup();
        driver = new ChromeDriver();
        driver.manage().window().maximize();
    }
    @AfterMethod
    public void tearDown() { if (driver != null) driver.quit(); }
}
```

```
public class TestListeners implements ITestListener {

    @Override
    public void onTestFailure(ITestResult result) {
        String testName = result.getName();              // which test failed
        TakesScreenshot ts = (TakesScreenshot) BaseTest.driver;
        File src = ts.getScreenshotAs(OutputType.FILE);
        File dest = new File("screenshots/" + testName + "_"
                             + System.currentTimeMillis() + ".png");
        try {
            FileUtils.copyFile(src, dest);
            // attach to Extent report as well:
            // ExtentManager.getTest().addScreenCaptureFromPath(dest.getPath());
        } catch (IOException e) { e.printStackTrace(); }
    }
}
```

Register it (either way):

```
@Listeners(TestListeners.class)     // on top of the test class
```

```
<listeners><listener class-name="listeners.TestListeners"/></listeners>   <!-- in testng.xml -->
```

Why this design is GOOD (say this): "The test classes have no screenshot logic at all — separation of concerns. If we change the report tool tomorrow, we change only the listener, not 200 tests."

MEMORY HOOK: *"Base holds the driver, the Listener hears the failure, the Screenshot is the proof."*

Q: How do you run tests in parallel?

Understand it: Add two attributes in testng.xml: `parallel` (what to split — tests/classes/methods) and `thread-count` (how many run at once).

Danger interviewers love: **each thread needs its OWN driver**, or tests grab each other's browser. Fix: keep the driver in a `ThreadLocal<WebDriver>` in BaseTest.

MEMORY HOOK: *"parallel + thread-count, and keep each driver ThreadLocal."*

Q: What is a flaky test and how do you fix it?

Understand it: A test that passes today, fails tomorrow, passes again — with NO code change. Causes: timing (element not ready), shared data clashing between tests, or a slow environment.

Fixes (say in this order): proper explicit waits -> isolated test data -> retry with IRetryAnalyzer -> then investigate the real cause; retry hides symptoms, it doesn't cure them.

MEMORY HOOK: *"Flaky = random failure; fix = better waits + retry + isolation."*

SECTION 10 — PAGE OBJECT MODEL IN DETAIL

Q: Show the full pattern in code.

```
public class LoginPage {
    WebDriver driver;

    @FindBy(id = "username")   WebElement txtUser;
    @FindBy(id = "password")   WebElement txtPass;
    @FindBy(css = "button[type='submit']") WebElement btnLogin;

    public LoginPage(WebDriver driver) {
        this.driver = driver;
        PageFactory.initElements(driver, this);   // connects @FindBy fields to real elements
    }

    public void login(String user, String pass) {
        txtUser.sendKeys(user);
        txtPass.sendKeys(pass);
        btnLogin.click();
    }
}

public class LoginTest extends BaseTest {
    @Test
    public void validLoginTest() {
        LoginPage lp = new LoginPage(driver);
        lp.login("admin", "admin123");
        Assert.assertTrue(driver.getTitle().contains("Dashboard"));
    }
}
```

Read the test class — it reads like a story, no locators anywhere. That's the whole point.

Q: PageFactory details interviewers check

- `PageFactory.initElements(driver, this)` in the constructor — the ritual. Elements are found **lazily** — not at page load, but the moment you first use each one.
- `@CacheLookup` — caches the element after first find -> faster, but breaks if the DOM re-renders -> use only for truly static elements.
- **Plain** `driver.findElement(By...)` **inside page classes is ALSO valid POM.** POM is the IDEA (one class per page), PageFactory is just one way to write it. Say this — it shows depth.

MEMORY HOOK: *"POM is the idea; PageFactory is one way to write it."*

Q: Senior-level extra: page components

Repeated widgets (header, footer, menu) become their own small classes (HeaderComponent) and page classes COMPOSE them. Shows you've worked on real frameworks.

SECTION 11 — COMMON EXCEPTIONS (memorize this table)

Exception	What it really means (plain words)	Typical fix
NoSuchElementException	"I searched, the element doesn't exist" — bad locator or page not loaded	fix locator / wait for visibility
StaleElementReferenceException	"I FOUND it, but the page re-rendered and my reference died" (AJAX refresh)	re-find the element; FluentWait ignoring Stale
ElementClickInterceptedException	"I tried to click but something else (spinner/ad/overlay) is on top"	wait for the overlay to vanish; JS click as plan B
ElementNotInteractableException	element exists but hidden or off-screen	scroll into view; wait for clickable
NoAlertPresentException	switched to an alert that isn't there	wait `alertIsPresent()` first
NoSuchFrameException / NoSuchWindowException	switched to a frame/window ID that doesn't exist	verify the handle exists first
TimeoutException	explicit wait ran out, condition never became true	longer wait or fix the condition
SessionNotFoundException	using the driver after quit()	create a new driver
InvalidSelectorException	your XPath/CSS has a syntax error	fix the syntax
WebDriverException	the parent of all the above	read the message

The classic trap — NoSuch vs Stale:

- NoSuch = element was **never born** (locator wrong).
- Stale = element was found, then the DOM refreshed and the old reference became a **dead link** — like having a friend's old phone number after he changed it.

MEMORY HOOK: *"NoSuch = never born; Stale = born but the page killed it."*

SECTION 12 — SCENARIO QUESTIONS CORNER (every scenario, fully explained)

Q: The element's ID changes on every page load. How do you handle it? (dynamic elements)

Understand it: Modern frameworks (React, Angular, JSF, Oracle ADF) auto-generate IDs like `id="input_3472"` — a fresh number every load. So your locator works today and dies tomorrow, and the test fails with `NoSuchElementException` even though the element is right there on the screen. The fix is never "run it again" — it is to stop using the unstable part and anchor on something that does not change: a stable parent, a label, a class, or the stable piece of the id itself.

What to check first: inspect the element, refresh the page, inspect again — see exactly WHICH part rotates (the number) and which part stays (`user_`). Also check whether the element has a stable `name`, `class`, `aria-label`, or `data-testid` you can use instead — the best dynamic-ID solution is often "don't use the id at all".

```
// XPath — match only the stable part
driver.findElement(By.xpath("//input[contains(@id,'user')]"));
driver.findElement(By.xpath("//button[starts-with(@id,'submit_')]"));

// CSS — the three operators (interviewers ask for these BY NAME)
driver.findElement(By.cssSelector("input[id^='user']"));    // ^= starts with
driver.findElement(By.cssSelector("button[id$='_btn']"));   // $= ends with
driver.findElement(By.cssSelector("div[id*='address']"));    // *= contains

// Best: anchor on a stable neighbour — labels rarely change
driver.findElement(By.xpath("//label[text()='Username']/following-sibling::input"));
```

MEMORY HOOK: *"The id is a chameleon — anchor on the rock next to it, or grab just its stable skin with contains/starts-with."*

SAY THIS: "When an ID is dynamic, I first compare the element across two page loads to find the stable part. Then I either match that stable part with XPath contains/starts-with or the CSS operators, or better, I anchor on a stable neighbour like the label. And long term I push the team to add stable data-testid attributes — locator stability is a team problem, not just a test problem."

Q: How do you handle CAPTCHA?

Understand it: CAPTCHA exists for exactly one purpose — to decide whether the visitor is a human. An automation tool solving it defeats the point, and any "solution" is a hack that breaks the moment the provider updates it. So the professional answer is not a trick — it is a testing-strategy answer, and that is precisely why interviewers love it: it shows you understand WHAT should be automated.

What the options actually are:

1. **Test-environment bypass (the real-world standard):** developers disable CAPTCHA in the test environment, or expose a backdoor that accepts a fixed token. The login FLOW is tested; the CAPTCHA service is the vendor's problem.
2. **Manual intervention window:** in a rare staging check, the framework pauses and a human types the code (usable, but it stops being automation).
3. **What we NEVER claim:** OCR libraries or CAPTCHA-solving services — saying this in an interview marks you as inexperienced (or worse).

SAY THIS: "CAPTCHA is designed to stop bots — me included — so I don't automate it. In test environments I ask developers to disable it or provide a bypass token, so I can still test the full login flow around it. It's a deliberate scope decision, not a limitation."

Q: How do you handle OTP / two-factor authentication?

Understand it: The OTP arrives OUTSIDE the browser — in a phone app or an SMS — so the browser session cannot see it. The testing problem is not "type the code", it is "GET the code into the test". Every real solution is about changing WHERE the code is delivered so the test can reach it.

The options, best first:

```
1. Test-mode OTP        — the app accepts a fixed code (e.g. 123456) in test envs
2. Read from the app's API/database — the same OTP is often stored server-side
   for verification; query it right before submitting (RestAssured / JDBC)
3. Dedicated test inbox — email OTPs read via JavaMail / an inbox API
4. TOTP automation       — for authenticator apps, generate the same time-based
   code with a library (e.g. java-totp) using the shared secret
```

```
// typical API read just before submit
String otp = given().get("/test-support/latest-otp?user=" + user).then().extract().path("code");
otpField.sendKeys(otp);
```

SAY THIS: "OTP lives outside the browser, so I change where it's delivered for tests: a fixed test-mode code if the app supports it, otherwise I fetch the OTP from the backend through an API or database call right before submitting. The goal is to keep the login flow automated end to end."

Q: How do you handle Shadow DOM elements?

Understand it: A shadow DOM is a sealed capsule inside a host element: the page shows you `<my-card>`, but the button inside it is invisible to normal locators — it lives on a separate shadow TREE. Encapsulation is why component libraries use it, and why a plain `findElement(By.xpath("//my-card//button"))` finds nothing. XPath cannot enter the capsule at all; only CSS can, and only after you open the door.

```
// Selenium 4 — enter ONE level at a time
WebElement host = driver.findElement(By.cssSelector("my-card"));
SearchContext shadow = host.getShadowRoot();          // the door key
shadow.findElement(By.cssSelector("button.delete")).click();

// nested roots: repeat — find the inner host INSIDE the shadow tree
WebElement innerHost = shadow.findElement(By.cssSelector("my-list"));
SearchContext innerShadow = innerHost.getShadowRoot();
innerShadow.findElement(By.cssSelector("li")).click();
```

The rules to recite: `getShadowRoot()` returns a `SearchContext` (not WebElement), only CSS works inside, XPath does not, and closed shadow roots are sealed for everyone.

MEMORY HOOK: *"Shadow DOM = a capsule. getShadowRoot() is the key — one capsule per turn, CSS only, XPath stays outside."*

SAY THIS: "For shadow DOM I take the host element, call getShadowRoot() — which returns a SearchContext — and search inside with CSS, because XPath cannot pierce shadow boundaries. For nested roots I repeat the pattern level by level. One more thing I mention in interviews: in Playwright the same problem disappears — CSS pierces open shadow roots automatically — which is one of the reasons teams migrate."

Q: How do you locate SVG icons?

Understand it: SVG elements live in a different XML namespace, so normal tag XPath like `//svg` silently matches nothing. You have to ask by `name()` — and the icon's visible meaning usually sits in a child element (`use`, `title`) or in a class, not in text.

```
// the canonical SVG locator
driver.findElement(By.xpath("//*[name()='svg']"));

// icon by its id/class (still through name())
driver.findElement(By.xpath("//*[name()='svg' and @class='delete-icon']"));

// clickable icon wrapping — often the svg sits inside the real button
driver.findElement(By.cssSelector("button svg"));
```

MEMORY HOOK: *"SVG is a foreigner in the DOM — you cannot call it by its plain tag name; you must ask for it by name()."*

SAY THIS: "SVG nodes are in a different XML namespace, so plain //svg doesn't match. I use `//*[name()='svg']`, usually combined with a class, or I target the button that wraps the icon."

Q: How do you read a specific table cell — say row 3, column 2?

Understand it: An HTML table is rows of cells — `tr` for row, `td` for cell. The interview version is "give me the cell AND give me the row by its content", and the content version is the one that survives data changes, because row 3 today is row 5 tomorrow.

```
// by position — row 3, column 2 (indexes count header rows too!)
driver.findElement(By.xpath("//table[@id='emp']/tbody/tr[3]/td[2]")).getText();

// by content — "the price cell of the Laptop row" (position-independent)
driver.findElement(
    By.xpath("//td[text()='Laptop']/following-sibling::td[1]")).getText();

// the whole row's text as one string
driver.findElement(By.xpath("//tr[td[text()='Laptop']]")).getText();
```

SAY THIS: "For a specific cell I use XPath indexes — table, tr, td — but I prefer locating the row by its content and reading its following-sibling cells, because content-based locators survive rows being added or reordered."

Q: How do you handle infinite scroll pages (feed keeps loading)?

Understand it: Infinite scroll replaces pagination: content loads as you approach the bottom, so the element list GROWS while you scroll. The stable strategy is a loop with an exit condition: scroll, wait for the count to grow, stop when it stops growing.

```
int oldCount = driver.findElements(By.cssSelector(".feed-item")).size();
while (true) {
    ((JavascriptExecutor) driver).executeScript("window.scrollTo(0, document.body.scrollHeight)");
    wait.until(d -> d.findElements(By.cssSelector(".feed-item")).size() > oldCount); // it grew
    int newCount = driver.findElements(By.cssSelector(".feed-item")).size();
    if (newCount == oldCount) break;   // nothing new came — we hit the bottom
    oldCount = newCount;
}
```

SAY THIS: "I scroll to the bottom with JavascriptExecutor, then wait until the element count increases, and I loop until two consecutive scrolls bring nothing new — that's the exit condition. Then I assert on the collected items."

Q: The element is VISIBLE but NOT clickable. Why, and what do you do?

Understand it: "Clickable" in Selenium's world is not "visible" — the element must also be in the viewport, not covered by anything, and enabled. When you see `ElementClickInterceptedException` or `ElementNotInteractableException`, one of these four thieves is at work, and the fix depends on WHICH one:

1. **Something covers it** — spinner, sticky header, cookie banner, modal. Wait for the cover to disappear first.
2. **It is below the fold** — present in the DOM, rendered, but outside the viewport. Scroll it into view.
3. **It is disabled** — `enabled` is false until some condition (form valid, terms accepted).
4. **It is still animating** — the element moves while Selenium computes the click point.

```
// 1. wait for the overlay to vanish, THEN click
wait.until(ExpectedConditions.invisibilityOf(overlay));
target.click();

// 2. scroll into view first
((JavascriptExecutor) driver).executeScript("arguments[0].scrollIntoView(true);", target);
target.click();

// 4. the honest last resort — click via JS (bypasses ALL actionability checks)
((JavascriptExecutor) driver).executeScript("arguments[0].click();", target);
```

When to admit the JS click: it works every time because it skips every check — which is exactly why it is a LAST resort: it hides real user-facing problems.

MEMORY HOOK: *"Clickable = visible + in-viewport + uncovered + enabled. Four thieves steal clicks: cover, fold, disabled, animation."*

SAY THIS: "Visible-but-not-clickable is almost always one of four causes: an overlay, the fold, a disabled state, or an animation. I read the exception first — intercepted means something's on top — then wait for the cover to vanish or scroll the element into view, and only if I need to prove a defect fast do I fall back to a JavaScript click, because it bypasses the very checks that prove a real user could click."

Q: How do you verify a tooltip?

Understand it: Tooltips come in two completely different species, and the test is different for each. The DEFAULT tooltip is just the `title` attribute — it is ALWAYS in the DOM, no hover needed. The CUSTOM tooltip (CSS/JavaScript) is a real element that only EXISTS after hover.

```
// species 1: the native title attribute — no hover needed
String tip = element.getAttribute("title");

// species 2: custom tooltip — hover brings it to life, THEN read it
new Actions(driver).moveToElement(element).perform();
wait.until(ExpectedConditions.visibilityOf(tooltipElement));
String customTip = tooltipElement.getText();
```

SAY THIS: "First I check which kind of tooltip it is. If it's the native title attribute, I just read getAttribute('title') — it's always in the DOM. If it's a custom CSS or JavaScript tooltip, I hover with the Actions class first, wait for it to become visible, and only then read its text."

Q: getAttribute() vs getText() — the difference people fumble?

Understand it: An element has two places where "text" can live: inside the tag's brackets (attributes) and between the opening and closing tags (visible text). Reading the wrong one is a classic live-coding bug — `getText()` on an input field always returns "", because a typed value is an ATTRIBUTE, not text.

```
<input id="user" value="jashu">Username</input>   <!-- (self-closing in reality) -->
```

```
driver.findElement(By.id("user")).getAttribute("value");  // "jashu" — the typed value
driver.findElement(By.id("user")).getText();               // "" — nothing BETWEEN the tags
driver.findElement(By.tagName("h1")).getText();           // the heading you SEE
```

MEMORY HOOK: *"Attribute lives IN the tag; Text lives BETWEEN the tags. Typed values are attributes — inputs have no text."*

SAY THIS: "getAttribute reads what's written inside the tag's brackets — value, href, title; getText reads the visible content between the tags. The practical one: whatever a user types into an input is the value attribute — getText on it returns empty."

Q: isDisplayed() vs isEnabled() vs isSelected()?

Understand it: Three different questions about an element's state — seeing, using, choosing — and mixing them up produces tests that pass while the app is broken.

Method	The question it answers	Typical element
`isDisplayed()`	is it rendered on screen right now?	error messages, spinners
`isEnabled()`	can the user actually use it? (not disabled)	buttons, inputs
`isSelected()`	is it chosen?	checkbox, radio, option

```
Assert.assertTrue(loginBtn.isEnabled(), "button greyed out with valid data!");
Assert.assertTrue(termsChk.isSelected(), "terms not accepted");
Assert.assertFalse(errMsg.isDisplayed(), "error shown while it should not be");
```

MEMORY HOOK: *"Displayed = seen; Enabled = usable; Selected = chosen — eyes, hands, choice."*

Q: How do you verify an element's color / CSS property?

Understand it: The color you SEE is not in an attribute — it comes from the stylesheet, so you read it with `getCssValue()`. And it comes back as an rgb/rgba string, so you compare against that exact string.

```
String color = button.getCssValue("color");          // "rgba(255, 0, 0, 1)"
Assert.assertEquals(color, "rgba(255, 0, 0, 1)");    // red — e.g. error state
String font  = button.getCssValue("font-size");      // "14px"
```

SAY THIS: "Visual properties live in CSS, not attributes, so I read them with getCssValue — for example getCssValue('color') — and compare against the rgb/rgba string the browser reports."

Q: How do you handle browser cookies in tests?

Understand it: Cookies are the session's memory. The two scenarios interviewers mean: (a) verify the app handles cookies correctly (add/delete), and (b) USE cookies to skip login — export once, inject forever.

```
// manipulate
driver.manage().addCookie(new Cookie("session", "abc123"));
driver.manage().deleteCookieNamed("session");
driver.manage().deleteAllCookies();

// the login-skip pattern: logged-in state reused across runs
Set<Cookie> all = driver.manage().getCookies();     // export after ONE real login
// in the next run: goto domain FIRST, inject, THEN refresh — cookies are per-domain
driver.get("https://app.com");
for (Cookie c : all) driver.manage().addCookie(c);
driver.navigate().refresh();                        // now logged in, no login test
```

SAY THIS: "Cookies are per-domain session state. I read them with getCookies, delete with deleteAllCookies for logout tests, and the powerful pattern is reusing a logged-in cookie session to skip repeated logins — though in Playwright this same idea is a first-class feature called storageState."

Q: A test PASSES headed but FAILS headless in CI. What do you check? (the classic — with the full checklist)

Understand it: Headless is not "the same browser minus the window" — it is a different ENVIRONMENT. There is no real screen, the viewport may differ, there is no visible cursor, GPU/animation rendering differs, and CI machines are slower. The same test therefore meets a DIFFERENT application experience. The failure is almost never "headless is broken" — it is one of the test's hidden assumptions (position, timing, viewport, hover) getting exposed. The interviewer is checking whether you DEBUG the failure or randomly change the test.

Check 1 — the viewport took your element away. In headed mode the window is normal; headless often starts smaller or with the old default. Elements can sit below the visible area, and a RESPONSIVE app can even render a different layout (a hamburger menu instead of the desktop form).

```
// rule 1: pin the window size so both modes meet the same app
driver.manage().window().setSize(new Dimension(1920, 1080));

// rule 2: from Chrome options — set it at birth
options.addArguments("--window-size=1920,1080");

// if the element is merely below the fold, scroll to it
((JavascriptExecutor) driver).executeScript("arguments[0].scrollIntoView(true);", element);
```

Check 2 — timing got exposed, not created. Headless often runs FASTER (no rendering to a screen), so a page that "had time" headed doesn't anymore. The fix is a targeted explicit wait for the actual condition — and never a blanket Thread.sleep.

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
WebElement button = wait.until(ExpectedConditions.elementToBeClickable(By.id("submit")));
button.click();
```

Check 3 — something covers the element. Popups, cookie banners, spinners and overlays behave the same headless, but they are the #1 cause of `ElementClickInterceptedException` in both modes. Wait for the cover to vanish, then click.

Check 4 — hover-dependent UI. Submenus that appear only after `moveToElement()` can behave differently without a real cursor. Verify the submenu is displayed before clicking into it.

```
new Actions(driver).moveToElement(menu).perform();
wait.until(ExpectedConditions.visibilityOf(submenuItem));
submenuItem.click();
```

Check 5 — JavaScript-rendered components. Many pages are built dynamically: page loads, JS runs, an API call happens, THEN the component appears. Waiting for page load is not waiting for the component — wait for the element's own condition.

Check 6 — take a screenshot AT the failure point (the listener from Section 9 already does this). One look answers "the login popup covers the button" or "the app rendered the mobile layout".

```
File src = ((TakesScreenshot) driver).getScreenshotAs(OutputType.FILE);
Files.copy(src.toPath(), Paths.get("failure.png"), StandardCopyOption.REPLACE_EXISTING);
```

Check 7 — read the actual exception before blaming headless. `NoSuchElementException` = wrong locator, not loaded, or different UI. `ElementClickInterceptedException` = covered. `ElementNotInteractableException` = hidden/off-screen. `TimeoutException` = the expected condition never became true. The exception is your first clue, not a decoration.

Check 8 — sanity-check the headless configuration itself.

```
ChromeOptions options = new ChromeOptions();
options.addArguments("--headless=new");          // the NEW headless — a real Chrome, no window
options.addArguments("--window-size=1920,1080");
WebDriver driver = new ChromeDriver(options);
```

`--headless=new` matters: old headless was a separate stripped browser; the new mode is the full Chrome engine without a window, and behaves far closer to headed. And the golden rule: the SAME test should pass in both modes — if it needs "headless-only" hacks, the test is hiding a real bug.

MEMORY HOOK: *"Headless = different world, not smaller world. Check in order: viewport, overlay, timing, hover — then screenshot the crime scene and read the exception before blaming headless."*

SAY THIS: "First I check the failure exception and the screenshot at the failure point — I never assume headless is guilty. Then I work through the usual culprits: viewport size and responsive layout, element visibility and scroll position, overlays covering the click, missing explicit waits on dynamic components, and hover-dependent menus. I also verify the headless options themselves — window-size pinned, --headless=new rather than the legacy mode. The goal is to find the exact reason from the evidence, because a test that only works headed is hiding a real condition my users will hit."

Q: How do you run different browsers without changing code? (browser from config)

Understand it: Hardcoding `new ChromeDriver()` in every test means every browser switch is a code change in a hundred places. The fix is one factory that reads the desired browser name from OUTSIDE the code — a properties file for local choice, a TestNG `@Parameter` for XML-driven cross-browser runs.

```
// BaseTest — the factory: read name, return the matching driver object
public static WebDriver createDriver(String browser) {
    if (browser.equalsIgnoreCase("chrome"))  return new ChromeDriver();
    if (browser.equalsIgnoreCase("firefox")) return new FirefoxDriver();
    if (browser.equalsIgnoreCase("edge"))    return new EdgeDriver();
    throw new IllegalArgumentException("Unknown browser: " + browser);
}

// local runs: browser comes from config.properties -> driver = createDriver(prop)
```

```
<!-- cross-browser runs: the SAME suite, once per browser -->
<test name="Chrome">
  <parameter name="browser" value="chrome"/>
  <classes><class name="com.qa.tests.LoginTest"/></classes>
</test>
<test name="Firefox">
  <parameter name="browser" value="firefox"/>
  <classes><class name="com.qa.tests.LoginTest"/></classes>
```

```
</test>
```

```
@BeforeMethod
public void setup(@Optional("chrome") @Parameters("browser") String browser) {
    driver = createDriver(browser);   // one method, any browser
}
```

MEMORY HOOK: *"The browser name travels (file or XML); the factory translates; the test never knows which browser it's driving."*

SAY THIS: "I never hardcode the driver. A factory method in the base class reads the browser name — from a properties file for local runs, or from a TestNG parameter defined in testng.xml for cross-browser execution — and returns the matching WebDriver object. The tests stay untouched; the XML decides the matrix."

SECTION 13 — MAVEN + CI/CD (they drill this after the framework)

13.1 The Maven script (memorize — this is exactly what to say)

> *"We use **Maven as our build and dependency management tool**. The main configuration file in our project is **pom.xml**. In pom.xml we define all the **dependencies** required for automation — Selenium WebDriver, TestNG, reporting libraries, and other utilities. Maven **automatically downloads these dependencies from the Maven repository** (Maven Central), so we don't have to manually download JARs and add them to the build path. We also use Maven to **compile the project and execute our TestNG test cases** — for example, running the suite with `mvn test`. And Maven follows a **standard project structure** — `src/main/java`, `src/test/java` — which keeps the project organized and consistent, so anyone joining the team knows exactly where everything is."*

MEMORY HOOK: *"Build + Dependencies + Structure = Maven's 3 jobs."*

13.1b Q: What are the THREE functionalities of Maven? (asked directly — know each one)

Understand each in detail — this is exactly what the interviewer wants when they ask "what does Maven do?":

Functionality 1 — BUILD management.

Maven compiles the code, runs the tests, and packages the output — through a defined **lifecycle** (`validate -> compile -> test -> package -> verify -> install -> deploy`). One command — `mvn clean test` — does what a whole script of javac/jar commands used to do. Every teammate and the CI server builds the SAME way, every time.

Functionality 2 — DEPENDENCY management.

You declare every external library (JAR) in pom.xml with its **GAV coordinates** (groupId : artifactId : version). Maven downloads them automatically from **Maven Central** into the local `.m2` repository, and even pulls THEIR dependencies (transitive dependencies). No manual JAR downloads, no build-path fixing, no "works on my machine" version drift — the same versions everywhere.

Functionality 3 — STANDARD project structure.

Maven enforces one fixed layout — `src/main/java` for main code, `src/test/java` for tests, `src/test/resources` for data/configs. This is "convention over configuration": any developer or QA opening ANY Maven project on Earth knows exactly where everything is — zero orientation time, and tools (IDEs, Jenkins, SonarQube) work with it out of the box.

SAY THIS: "Maven gives us three things: it's a build tool — one command compiles, tests and packages through a fixed lifecycle; it's a dependency manager — we declare JARs as GAV coordinates in pom.xml and Maven pulls them from Maven Central with transitive dependencies, so no manual JARs; and it enforces the standard project structure — src/main/java, src/test/java — so every Maven project is organized the same way and CI tools plug in without configuration."

MEMORY HOOK: *"Builds the code, Fetches the JARs, Fixes the layout — B-F-F."* (Build, Fetch, Fixed structure.)

13.2 pom.xml dependencies with names + versions (know these numbers)

```
<dependencies>
    <!-- Selenium — browser automation -->
    <dependency>
        <groupId>org.seleniumhq.selenium</groupId>
        <artifactId>selenium-java</artifactId>
        <version>4.45.0</version>
    </dependency>

    <!-- TestNG — the test framework -->
    <dependency>
        <groupId>org.testng</groupId>
        <artifactId>testng</artifactId>
        <version>7.12.0</version>
        <scope>test</scope>
    </dependency>

    <!-- WebDriverManager — downloads driver .exe files automatically -->
    <dependency>
        <groupId>io.github.bonigarcia</groupId>
        <artifactId>webdrivermanager</artifactId>
        <version>5.9.2</version>
    </dependency>

    <!-- Extent Reports — HTML reporting with screenshots -->
    <dependency>
        <groupId>com.aventstack</groupId>
        <artifactId>extentreports</artifactId>
        <version>5.1.2</version>
    </dependency>

    <!-- Apache POI — read/write Excel test data -->
    <dependency>
        <groupId>org.apache.poi</groupId>
        <artifactId>poi-ooxml</artifactId>
        <version>5.3.0</version>
    </dependency>

    <!-- Log4j2 — logging -->
    <dependency>
        <groupId>org.apache.logging.log4j</groupId>
        <artifactId>log4j-core</artifactId>
        <version>2.23.1</version>
    </dependency>

    <!-- Commons-io — file utilities (copying screenshot files) -->
    <dependency>
        <groupId>commons-io</groupId>
        <artifactId>commons-io</artifactId>
        <version>2.16.1</version>
    </dependency>
</dependencies>
```

And the build plugin every pom needs (this answers "how does `mvn test` know to run testng.xml?"):

```
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-plugin</artifactId>
      <version>3.5.4</version>
      <configuration>
        <suiteXmlFiles>
          <suiteXmlFile>testng.xml</suiteXmlFile>
        </suiteXmlFiles>
      </configuration>
    </plugin>
  </plugins>
</build>
```

The **maven-surefire-plugin** is what the `test` phase calls — pointing it at testng.xml is how `mvn test` runs the whole TestNG suite.

Report alternative to know: **Allure** — `io.qameta:allure-testng:2.29.x` plus the allure-maven plugin. Many teams use it instead of (or alongside) Extent — its adapter hooks TestNG listeners automatically and produces the famous step-by-step report.

How versions behave (so you're never trapped): Selenium releases almost monthly (it is on the 4.4x line in 2026); TestNG is on the 7.12 line. If the interviewer asks "which version?" — state the number, then add: *"in the project we keep updating to the latest stable from mvnrepository.com."* What really matters is knowing the **groupId : artifactId : version** (GAV) format.

MEMORY HOOK for the list: Selenium, TestNG, WebDriverManager, Extent, POI, Log4j, Commons -> *"Some Tests Want Evidence, POI Logs Clicks."*

13.3 Maven concepts you must be able to explain

- **What Maven actually does** — 3 jobs: builds the project, downloads/manages the JARs, enforces the standard folder layout.
- **pom.xml** — "Project Object Model" — the single file where dependencies, plugins, and build settings live.
- **Where do the JARs go?** Maven downloads them from **Maven Central** into a local `.m2` folder on your machine — first build is slow, later builds reuse the cache.
- **Scope** `test` — that JAR is available only to test code; `compile` (default) — everywhere.
- **Build lifecycle (memorize the chain):** `validate -> compile -> test -> package -> verify -> install -> deploy`.

MEMORY HOOK: *"Very Cool Tests Produce Very Impressive Deployments."*

- **Daily commands:** `mvn clean` (delete target/), `mvn compile`, `mvn test`, `mvn clean test`, `mvn dependency:tree` (debug JAR conflicts).
- **maven-surefire-plugin** — the plugin that finds testng.xml and runs the suite during the `test` phase. Senior line: "We configure surefire in pom.xml to point at our testng.xml, so `mvn test` runs the whole suite."
- **Why Maven instead of manual JARs?** Without Maven, every teammate and the CI server must download the same JARs by hand, and versions drift. With Maven, `mvn test` resolves everything automatically — same versions everywhere.

13.4 The CI/CD chain (say as one flow)

SAY THIS: "Code is pushed to **Git** -> the **Jenkins** job triggers (on commit or nightly) -> runs `mvn clean test` with testng.xml -> Jenkins publishes the **Extent/Allure HTML report** -> result notification goes by email/Slack."

MEMORY HOOK: *"Push -> Trigger -> Build -> Test -> Report -> Notify."*

- **Extent Reports** — HTML report with pass/fail/skip, screenshots, dashboards.
- **Log4j2** — logs each step; levels TRACE < DEBUG < INFO < WARN < ERROR.
- **Selenium Grid** — run in parallel across many browsers/machines; Grid 4 runs nicely with Docker.

SECTION 14 — JAVA ESSENTIALS ASKED ALONGSIDE SELENIUM (each with code you can speak from)

Q: How do the OOP pillars apply to your framework? (they want Selenium mapping, not theory)

Understand it: Nobody asks "define encapsulation" to a Selenium tester — they ask "where is encapsulation in YOUR framework". The pillars must be spoken as design decisions you made, each with a live example from the framework.

1. **Encapsulation** — locators live `private` inside a page class; the test cannot touch them, only the page's public methods. A locator change touches ONE file.

```
public class LoginPage {
    private final By username = By.id("user");     // private — hidden from tests
    public void login(String u, String p) { ... } // public — the only door in
}
```

2. **Inheritance** — tests extend `BaseTest`, so every test inherits driver setup, waits, and teardown for free. Write the plumbing ONCE.
3. **Polymorphism** — the star answer: `WebDriver driver = new ChromeDriver();`. The reference type is the INTERFACE, the object is the real browser driver. Runtime polymorphism is exactly why the cross-browser factory works — one line changes Chrome to Firefox.

```
WebDriver driver = new ChromeDriver();   // interface reference, Chrome object
driver = new FirefoxDriver();            // same variable, different animal
```

4. **Abstraction** — `driver.findElement()` is a black box: you call it, WebDriverImpl + the browser driver do the W3C protocol dance underneath. You use the WHAT without the HOW.

MEMORY HOOK: *"Encapsulation hides locators; Inheritance shares plumbing; Polymorphism swaps browsers; Abstraction hides the protocol."*

SAY THIS: "In my framework encapsulation is the POM itself — locators are private inside page classes. Inheritance is BaseTest — setup and teardown written once. Polymorphism is the line every interviewer likes: WebDriver driver = new ChromeDriver() — the interface reference that lets me switch browsers with one line. And abstraction is what WebDriver itself is — I call findElement and the W3C protocol happens behind the curtain."

Q: Overloading vs Overriding? (compile time vs runtime — say it with a test example)

Understand it: Same method name, two different games. Overloading = same name, DIFFERENT parameter lists, in the same class — the compiler picks the version at compile time. Overriding = the CHILD re-defining the parent's exact method — the JVM picks the version at RUNTIME based on the actual object.

```
// OVERLOADING — same name, different shape (compile time)
public void login(String user, String pass) { ... }
public void login(String user, String pass, boolean rememberMe) { ... }

// OVERRIDING — child replaces parent's exact method (runtime)
class BaseTest {
    protected void beforeHook() { System.out.println("generic setup"); }
}
class SmokeBaseTest extends BaseTest {
    @Override protected void beforeHook() { System.out.println("smoke setup + data seed"); }
}
new SmokeBaseTest().beforeHook();   // "smoke setup + data seed" — the object wins
```

MEMORY HOOK: *"Overload = different shape, same class, compiler decides. Override = child replaces parent, object decides."*

Q: final vs finally vs finalize? (the classic trap triple)

Understand it: Three completely unrelated things that just share six letters — which is why interviewers ask it as a filter.

1. **final** — a lock: the variable can't be reassigned, the method can't be overridden, the class can't be extended. In frameworks: `private final By username = ...` — a locator must never change mid-run.

```
private final WebDriver driver;        // set once in the constructor, forever
```

2. **finally** — the block after try/catch that ALWAYS runs — where the driver quits so the browser closes even after a crash.

```
try { runTest(); } catch (Exception e) { log(e); }
finally { driver.quit(); }             // runs on success, failure, and crash
```

3. **finalize()** — the garbage collector's pre-destruction hook. Deprecated in modern Java — say "legacy, we never rely on it; I quit my drivers explicitly in finally."

MEMORY HOOK: *"final = lock it, finally = always run it, finalize = forget it."*

Q: String vs StringBuilder — why does it matter in tests?

Understand it: A String in Java is IMMUTABLE — every "change" secretly builds a brand new object and throws the old one away. In a loop (building XPaths, concatenating data), that is hundreds of throwaway objects; StringBuilder mutates ONE buffer in place.

```
// BAD — every += builds and discards a whole new String
String xpath = "//div[";
for (int i = 0; i < 20; i++) xpath += i + "]/div[";

// GOOD — one buffer, mutated in place
StringBuilder sb = new StringBuilder("//div[");
for (int i = 0; i < 20; i++) sb.append(i).append("]/div[");
String xpath = sb.toString();
```

SAY THIS: "Strings are immutable, so loop concatenation creates garbage objects every iteration — in data-driven tests that's real memory and time. I use StringBuilder whenever a string is built in a loop, and String only for fixed values."

Q: Which collections do you actually use in automation? (List, Set, Map — each with a REAL use)

Understand it: The interviewer wants the collection tied to a Selenium scenario, not a Java textbook definition.

Collection	Rule	The Selenium moment
List (`ArrayList`)	ordered, duplicates allowed	`findElements()` returns it — 10 matching links is a List
Set (`HashSet`)	no duplicates	`driver.getWindowHandles()` — each window exists once
Map (`HashMap`)	key -> value	test data: `{"username" -> "jashu", "password" -> "..."}` from Excel/JSON

```
List<WebElement> links = driver.findElements(By.tagName("a"));   // duplicates fine
Set<String> windows = driver.getWindowHandles();                  // unique by nature
Map<String, String> row = excelReader.nextRowAsMap();             // column -> value
links.stream().filter(l -> l.isDisplayed()).count();              // streams on top
```

Q: Exception handling — checked vs unchecked, and where each appears in your framework?

Understand it: Checked exceptions are forced on you at COMPILE time (file I/O, SQL) — the compiler says "handle or declare". Unchecked (runtime) exceptions (NullPointer, StaleElement) appear only when they blow up. A QA answer must name framework examples.

```
// CHECKED — the compiler demands it: reading Excel
try {
    FileInputStream fis = new FileInputStream("data.xlsx");
} catch (IOException e) {                       // handle
    throw new RuntimeException("Test data missing: " + e.getMessage());  // or wrap
}

// UNCHECKED — appears at runtime; Selenium's own exceptions are unchecked
// NoSuchElementException, StaleElementReferenceException — all extend RuntimeException
```

SAY THIS: "Checked exceptions like IOException happen at compile time — my Excel and file-handling code wraps them into a RuntimeException with a readable message. Unchecked ones like NoSuchElementException are runtime by design — I don't catch them, I WAIT for conditions so they never occur, and I use try/catch/finally with driver.quit() in finally so no browser leaks on a crash."

Q: == vs .equals()? (the one that separates Java people from script kiddies)

```
String a = new String("admin");
String b = new String("admin");
a == b               // false — two different objects (reference comparison)
a.equals(b)          // true  — same characters (content comparison)

// the Selenium moment — comparing expected vs actual text:
Assert.assertEquals(page.getConfirmationText(), "Order placed");  // .equals inside
Assert.assertTrue(actual == expected);   // WRONG for strings — reference trap
```

MEMORY HOOK: *"== asks 'same object?', .equals() asks 'same content?' — tests compare content, always."*

Q: Where do you use Java 8 features in tests? (lambda + streams)

Understand it: ExpectedConditions used to need clunky anonymous classes; lambdas made waits one line, and streams made element lists filterable like SQL.

```
// lambda inside a wait — the condition IS the lambda
wait.until(d -> d.findElement(By.id("status")).getText().contains("Done"));

// streams over a found list — collect link texts, filter, verify
List<WebElement> links = driver.findElements(By.cssSelector(".product a"));
boolean hasLaptop = links.stream()
    .map(WebElement::getText)               // method reference
    .anyMatch(t -> t.contains("Laptop"));

links.stream().filter(WebElement::isDisplayed).count();
```

SAY THIS: "Lambdas make my explicit waits one-liners — until this lambda over the driver — and streams handle bulk element checks: map to text, filter for displayed, anyMatch for a contains-check, without writing a single for-loop."

SECTION 15 — CODING QUESTIONS THEY MAKE YOU WRITE LIVE (full code — practice until each is 5 minutes)

These are the ten they put in front of you with a blank editor. The code below is the complete answer each time — read it, then retype it from memory.

15.1 Login test with POM + TestNG + DataProvider (the #1 hands-on — own it cold)

What they are checking: can you separate page from test, drive it with data, and assert meaningfully — all three layers at once.

```
// ---------- LoginPage.java — the POM layer ----------
public class LoginPage {
    private final WebDriver driver;
    private final By username = By.id("username");
    private final By password = By.id("password");
    private final By loginBtn = By.cssSelector("button[type='submit']");
    private final By errorMsg = By.cssSelector(".alert-danger");

    public LoginPage(WebDriver driver) { this.driver = driver; }

    public void login(String user, String pass) {
        driver.findElement(username).sendKeys(user);
        driver.findElement(password).sendKeys(pass);
        driver.findElement(loginBtn).click();
    }
    public String getError() {
        return driver.findElement(errorMsg).getText();
    }
}

// ---------- LoginTest.java — the test layer ----------
public class LoginTest extends BaseTest {
    LoginPage lp;

    @BeforeMethod
    public void pageSetup() { lp = new LoginPage(driver); }

    @Test(dataProvider = "creds")
    public void loginTest(String user, String pass, String expected) {
        lp.login(user, pass);
        if (expected.equals("pass"))
            Assert.assertEquals(driver.getTitle(), "Dashboard");
        else
            Assert.assertTrue(lp.getError().contains(expected));
    }

    @DataProvider(name = "creds")
    public Object[][] creds() {
        return new Object[][] {
            {"admin",   "admin123", "pass"},
            {"admin",   "wrong",    "Invalid credentials"},
            {"locked",  "any",      "account is locked"}
        };
    }
}
```

Why it scores: locators private (encapsulation), data in the provider, both positive and negative rows, and the object array's third column drives the assertion — that's a framework thinker, not a scripter.

15.2 Explicit wait until clickable, then click

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
WebElement button = wait.until(ExpectedConditions.elementToBeClickable(By.id("submit")));
button.click();

// one line if you like it tight:
new WebDriverWait(driver, Duration.ofSeconds(10))
    .until(ExpectedConditions.elementToBeClickable(By.id("submit"))).click();
```

Say while writing: "elementToBeClickable covers visible + enabled — I wait for the condition, not the clock, so no Thread.sleep ever appears in my code."

15.3 Switch between two windows and return to the parent

```
String parent = driver.getWindowHandle();          // remember home
driver.findElement(By.id("openWin")).click();     // popup opens

for (String handle : driver.getWindowHandles()) { // handles are a Set
    if (!handle.equals(parent)) {
        driver.switchTo().window(handle);         // now driving the popup
        break;
    }
}
Assert.assertEquals(driver.getTitle(), "Offers");

driver.switchTo().window(parent);                  // come home
Assert.assertEquals(driver.getTitle(), "Home");
```

The line to say: "getWindowHandle singular is THIS window; plural returns the Set of all — I loop it, skip the parent, switch to the new one."

15.4 Count and print all links on a page

```
List<WebElement> links = driver.findElements(By.tagName("a"));
System.out.println("Total links: " + links.size());

links.stream()
     .map(l -> l.getAttribute("href"))     // href is an ATTRIBUTE, not text!
     .filter(Objects::nonNull)
     .distinct()
     .forEach(System.out::println);
```

The two traps they watch for: `findElements` (plural, never throws for zero matches), and href read with `getAttribute`, not `getText`.

15.5 Select a dropdown value in all three ways

```
Select sel = new Select(driver.findElement(By.id("country")));
sel.selectByVisibleText("India");   // what the user sees — the stable choice
sel.selectByValue("in");            // the value attribute
sel.selectByIndex(2);               // position — the fragile one
```

Say while writing: "This only works for a real SELECT tag. A div/ul 'custom dropdown' needs a click to open, then a locator on the option — Select throws UnexpectedTagNameException on fakes."

15.6 Screenshot on failure using a listener (condensed — full flow in Section 9)

```
public class TestListener implements ITestListener {
    @Override
    public void onTestFailure(ITestResult result) {
        WebDriver driver = ((BaseTest) result.getInstance()).driver;  // pull the driver
        File src = ((TakesScreenshot) driver).getScreenshotAs(OutputType.FILE);
        try {
            FileUtils.copyFile(src,
                new File("./screenshots/" + result.getName() + ".png"));
        } catch (IOException e) { e.printStackTrace(); }
    }
}
// wire: @Listeners(TestListener.class) above the test class
```

The flow to narrate: BaseTest holds the driver -> the listener's onTestFailure fires -> pull the driver from result.getInstance() -> cast TakesScreenshot -> copy the file. Four steps, in that order, every time.

15.7 A login form sitting inside an iframe

```
driver.switchTo().frame("loginFrame");            // by name/id
// driver.switchTo().frame(0);                   // by index — brittle
// driver.switchTo().frame(webElement);           // by element

driver.findElement(By.id("username")).sendKeys("admin");
driver.findElement(By.id("password")).sendKeys("secret");
driver.findElement(By.id("login")).click();

driver.switchTo().defaultContent();               // LEAVE the frame — always
```

The trap: every switchTo().frame() needs a matching exit — defaultContent() or parentFrame() — or the next test's locators search inside a dead frame.

15.8 Excel-driven DataProvider with Apache POI (the full pattern)

```
public class ExcelUtil {
    @DataProvider(name = "excelData")
    public Object[][] readExcel() throws IOException {
        FileInputStream fis = new FileInputStream("testdata/logins.xlsx");
        Workbook wb = new XSSFWorkbook(fis);              // .xlsx
        Sheet sheet = wb.getSheet("Sheet1");

        int rows = sheet.getLastRowNum();                 // excludes header
        Object[][] data = new Object[rows][];
        for (int i = 1; i <= rows; i++) {                  // start at 1: skip header
            Row row = sheet.getRow(i);
            data[i - 1] = new Object[] {
                row.getCell(0).getStringCellValue(),
                row.getCell(1).getStringCellValue()
            };
        }
        wb.close();
        return data;
    }
}
```

The three details they probe: `getLastRowNum` vs `getPhysicalNumberOfRows` (off-by-one header trap), start the loop at row 1, and XSSF (not HSSF — that's the old .xls).

15.9 SoftAssert all form validation messages

```
@Test
public void formValidations() {
    driver.findElement(By.id("submit")).click();   // submit EMPTY form
    SoftAssert soft = new SoftAssert();

    soft.assertTrue(driver.findElement(By.id("errUser")).isDisplayed(),
                    "username error missing");
    soft.assertTrue(driver.findElement(By.id("errPass")).isDisplayed(),
                    "password error missing");
    soft.assertTrue(driver.findElement(By.id("errTerms")).isDisplayed(),
                    "terms error missing");

    soft.assertAll();   // forget this line and the test SILENTLY PASSES
}
```

15.10 IRetryAnalyzer — retry failed tests automatically

```
public class RetryAnalyzer implements IRetryAnalyzer {
    private int attempt = 0;
    private static final int MAX = 2;                       // 2 retries

    @Override
    public boolean retry(ITestResult result) {
        return attempt++ < MAX;    // true = run it again
    }
}

// wire it to a test:
@Test(retryAnalyzer = RetryAnalyzer.class)
public void flakyCheckout() { ... }
```

The follow-up they always ask: "Isn't retry hiding bugs?" -> "Retries are for FLAKINESS, not failures — a real defect fails all three attempts. And a test that passes only on retry is marked 'flaky' in the report, so we still burn it down."

SECTION 16 — RAPID-FIRE, FULLY EXPLAINED (final revision — every line now with its why)

How to use this section: each line is the one-sentence answer, followed by the explanation you give when the interviewer says "explain that a bit more". Cover the paragraph, speak the first line.

- **Selenium talks to browsers through driver binaries using the W3C protocol.**

  Your Java code never touches the browser — it sends W3C HTTP commands to a driver exe (chromedriver, geckodriver), and THAT exe drives the browser. Since Selenium 4 the protocol is the W3C worldwide standard, so the same command works on every browser (before, each browser spoke its own "dialect" — the JSON Wire Protocol — causing browser-specific weirdness).

- **ID is the fastest locator; XPath is the most powerful — last resort.**

  ID lookup is O(1) — the browser's internal hash map goes straight to it. XPath (especially the // descendant scan) walks the DOM tree. So: ID -> name/class/CSS -> XPath only when nothing semantic is left. And when you do use XPath, use RELATIVE (`//`) — absolute `/html/body/div[2]/...` breaks with any layout change.

- **Relative XPath starts with //, absolute with a single /.**

  `//` means "anywhere in the document — find every match"; a single `/` means "start from the html root and follow EXACTLY this path". Absolute paths are the most fragile locators in the world — one div reorder and the test dies.

- **Implicit = global blanket; explicit = targeted condition; fluent = explicit with custom polling.**

  Implicit wait is set ONCE and silently applies to every findElement — it waits for elements to APPEAR, nothing else, and it can't wait for visibility, clicks, or disappearances. Explicit wait is one targeted condition on one element (`elementToBeClickable`). Fluent is explicit with knobs — polling interval, exceptions to ignore (`ignoring(StaleElementReferenceException.class)`). Never mix implicit with explicit — the wait times add up unpredictably.

- **presence = in DOM; visibility = rendered; clickability = visible + enabled.**

  An element can be present (in the HTML) but invisible (`display:none`) — and present + visible but not clickable (covered/disabled). Each has its own ExpectedCondition: `presenceOfElementLocated`, `visibilityOfElementLocated`, `elementToBeClickable` — pick the one that matches what the USER actually needs, not just "it exists".

- **close() one window; quit() ends the whole session.**

  `close()` shuts the current window only — if other windows are open the session (and chromedriver.exe process) stays alive. `quit()` kills the session, all windows, and the driver process. That's why tearDown uses quit() — close() leaks driver processes on the CI machine (you'll see 40 chromedriver.exe in Task Manager).

- **getWindowHandle = this window; getWindowHandleS = all windows.**

  Singular returns ONE string — the current window's unique id. Plural returns a Set of ALL open window ids (a Set because each id is unique). The classic pattern: save the parent handle, open popup, loop the Set, switch to the one that isn't parent.

- **Frames need switchTo; alerts need switchTo().alert(); OS dialogs need Robot.**

  A frame is a page-inside-a-page — WebDriver must be "inside" it to see its elements. An alert is browser-owned — no locators at all, only `alert.accept()` / `dismiss()` / `sendKeys()`. A Windows file dialog is OS-owned — the browser can't see it at all, so you either avoid it (sendKeys the path straight to the input) or drive it with the Robot class keystrokes.

- **Actions = mouse; without perform() nothing happens.**

  The Actions class BUILDS a gesture sequence — moveToElement, click, dragAndDrop — and only `perform()` executes it. build() creates the action, perform() runs it. Forgetting perform() is the #1 Actions bug: no error, nothing happens, test "passes" silently.

- **TestNG setup order: Suite > Test > Class > Method; teardown exact reverse.**

Hotel metaphor: enter the hotel (Suite), the floor (Test), the room (Class), then each task (Method) — you leave in reverse order: method, class, test, suite. Setup cascades top-down; teardown cascades bottom-up.

- **Without priority, tests run alphabetically.**

  TestNG doesn't promise code order — it sorts method NAMES alphabetically. `testB_addToCart` runs before `testA_login` if you let it. Control it with `priority = 1` (lowest first) or `dependsOnMethods` for real dependencies — but prefer tests that don't depend on each other at all.

- **Hard assert halts; soft assert collects everything and speaks at assertAll().**

  `Assert.assertEquals` throws immediately — the rest of the test never runs. `SoftAssert` records every failure quietly and reports ALL of them when you call `assertAll()` — which you must, or the test passes with failures silently collected. Use soft when you want the full picture (all validation messages), hard when the next step depends on this one.

- **DataProvider = many rows in code; Parameters = one value from XML.**

  A DataProvider method RETURNS an Object[][] — the test runs once per ROW, inside Java. `@Parameters` injects single values (like browser name) from testng.xml — configuration, not test data. Rule of thumb: data that belongs to the test = DataProvider; environment config = Parameters.

- **NoSuch = never found; Stale = found, then the page re-rendered.**

  `NoSuchElementException`: the locator matched nothing — wrong locator or too early (fix: verify locator, wait for visibility). `StaleElementReferenceException`: you FOUND it, then an AJAX refresh replaced the DOM node — your reference points to a dead element, like dialing a friend's old number after he changed it. Fix: re-find the element, or FluentWait ignoring Stale.

- **POM is the pattern; PageFactory is one implementation of it.**

  Page Object Model = the DESIGN IDEA — one class per page, locators + behavior together, tests stay clean. PageFactory is the `@FindBy` + `initElements` helper that builds locators lazily — you can do POM perfectly well without it. POM is the recipe; PageFactory is one brand of mixer.

- **Listener: Base holds the driver, onTestFailure hears the failure, TakesScreenshot is the proof.**

  The listener runs OUTSIDE the test class — it has no `driver` variable of its own. So the flow is: pull the driver from `result.getInstance()` (the test object), the `onTestFailure` callback fires automatically, and the cast `((TakesScreenshot) driver)` captures the evidence. One line each — that's why it's recitable.

- **CI flow: Push -> Trigger -> Build -> Test -> Report -> Notify.**

  You push code; the pipeline triggers (webhook), Maven builds (`mvn clean test`), surefire runs testng.xml, reports are published (Extent/Allure/HTML), and the team is notified — pass keeps merging, fail blocks the merge. The suite runs nightly AND on every PR — regressions surface within a day, not next month.

SECTION 17 — CLOSING ANSWER + REVISION PLAN

Q: "What is the strength of your framework?"

> *"Maintainability and stability. Locators change in ONE place thanks to POM. Test data sits outside the code in Excel/properties, so coverage grows without code changes. Explicit waits and retry logic control flakiness. Extent screenshots make failure analysis fast. And Jenkins runs the whole suite nightly, so a regression is caught within a day — the framework scales from one browser to Grid parallel runs without touching the test code."*

MEMORY HOOK: *"One place, one data file, one command."*

Last-day revision plan

1. Speak the 60-second framework script out loud 5 times.
2. Recite "Can Big Projects Test Until Reports Come" and draw the tree from memory.
3. Write the TestNG annotation order from memory (hotel: Suite > Test > Class > Method).
4. Revise the exceptions table — especially Stale vs NoSuch.
5. Read Section 16 twice the night before.

Walk in like the framework is YOURS — after this guide, it is.

SECTION 18 — THE REMAINING CORNER (extras interviewers check — don't get caught here)

18.1 Browser options deep-dive + SSL certificate handling (classic question)

Q: How do you handle the SSL certificate error page ("Your connection is not private")?

Understand it: The test browser shows a warning page instead of the site. You tell Chrome to accept insecure certs in the OPTIONS:

```
ChromeOptions options = new ChromeOptions();
options.setAcceptInsecureCerts(true);            // THE answer
options.addArguments("--headless=new");          // run without a visible window (CI)
options.addArguments("--start-maximized");
options.addArguments("--disable-notifications");  // kill "allow notifications" popups
options.addArguments("--incognito");
options.setBinary("path/to/chrome");             // non-default Chrome location
WebDriver driver = new ChromeDriver(options);
```

Download-folder control lives in prefs:

```
HashMap<String, Object> prefs = new HashMap<>();
prefs.put("download.default_directory", System.getProperty("user.dir") + "/downloads");
options.setExperimentalOption("prefs", prefs);
```

MEMORY HOOK: *"SSL = setAcceptInsecureCerts(true) — it's an OPTION, not a wait."*

18.2 The THREE Selenium timeouts (a favorite trick question)

Timeout	Controls	Code
implicit wait	element FIND operations	`timeouts().implicitlyWait(10, SECONDS)`
pageLoadTimeout	how long to wait for a PAGE to load	`timeouts().pageLoadTimeout(30, SECONDS)`
setScriptTimeout	async JavaScript execution time	`timeouts().setScriptTimeout(10, SECONDS)`

MEMORY HOOK: *"Page, Script, Element — three clocks, all set on* `driver.manage().timeouts()`*."*

18.3 Singleton and Factory patterns in the framework (asked in senior rounds)

Understand it — the problem: in parallel runs, tests must not grab each other's driver. Two patterns fix it:

- **Singleton DriverManager** — one `getDriver()` entry point; combined with **ThreadLocal** so each THREAD gets its own single driver:

```
public class DriverManager {
    private static ThreadLocal<WebDriver> driver = new ThreadLocal<>();
    public static WebDriver getDriver() {
        if (driver.get() == null) {
            driver.set(new ChromeDriver());   // created once per thread
        }
        return driver.get();
    }
    public static void quitDriver() {
        if (driver.get() != null) { driver.get().quit(); driver.remove(); }
    }
}
```

- **Factory pattern (BrowserFactory)** — one method returns the right driver from the browser name; tests never call `new ChromeDriver()` directly:

```
public static WebDriver create(String browser) {
    return switch (browser.toLowerCase()) {
        case "chrome"  -> new ChromeDriver();
        case "firefox" -> new FirefoxDriver();
        case "edge"    -> new EdgeDriver();
        default -> throw new IllegalArgumentException("Unknown browser");
    };
```

```
}
```

MEMORY HOOK: *"Singleton = one per thread, Factory = pick by name."*

18.4 Apache POI — reading Excel (they make you WRITE this)

```
FileInputStream fis = new FileInputStream("testdata/Data.xlsx");
Workbook wb = new XSSFWorkbook(fis);            // XSSF = .xlsx, HSSF = .xls
Sheet sheet = wb.getSheet("LoginData");
for (int i = 1; i <= sheet.getLastRowNum(); i++) {   // start at 1 — skip the header
    String user = sheet.getRow(i).getCell(0).getStringCellValue();
    String pass = sheet.getRow(i).getCell(1).getStringCellValue();
}
wb.close(); fis.close();
```

Feeding a DataProvider: build and return `Object[sheet.getLastRowNum()][2]` inside the loop. Tip: `WorkbookFactory.create(fis)` auto-detects .xls vs .xlsx.

MEMORY HOOK: *"Workbook > Sheet > Row > Cell — POI is the Excel map."*

18.5 Extent Reports — the actual setup code

```
// @BeforeSuite
ExtentSparkReporter spark = new ExtentSparkReporter("reports/extent.html");
ExtentReports extent = new ExtentReports();
extent.attachReporter(spark);

// in the listener's onTestFailure / onTestSuccess
ExtentTest test = extent.createTest(result.getMethod().getMethodName());
test.log(Status.PASS, "Test passed");
test.log(Status.FAIL, "Failed: " + result.getThrowable());
test.addScreenCaptureFromPath("screens/" + testName + ".png");

// @AfterSuite — CRITICAL: without flush the report file is EMPTY
extent.flush();
```

MEMORY HOOK: *"Create -> attach -> createTest -> log -> FLUSH. No flush, no report."*

18.6 Cookies in detail

```
driver.manage().addCookie(new Cookie("session", "abc123"));
Set<Cookie> all = driver.manage().getCookies();
driver.manage().deleteCookieNamed("session");
driver.manage().deleteAllCookies();
```

Interview use case: **inject the auth cookie instead of logging in through the UI in every test** — the Selenium-era ancestor of Playwright's storageState idea.

18.7 API and Database testing alongside Selenium

- **RestAssured (API):** `given().when().get("/api/users/1").then().statusCode(200);` — say: "We create test data via API instead of slow UI forms, run the UI flow, then clean up via API."
- **JDBC (Database):** `Connection c = DriverManager.getConnection(url, user, pass); ResultSet rs = c.createStatement().executeQuery("select status from orders where id=1");` — say: "When the UI shows a success toast, I verify the row actually landed in the DB — the UI proves the front, the DB proves the truth."

18.8 BDD / Cucumber (asked in a huge share of interviews)

Understand it: BDD = Behavior-Driven Development — tests written as plain-English behaviors that product owners can read. Gherkin syntax:

```
Feature: Login
  Scenario: Valid login
    Given user is on the login page
    When user enters valid credentials
    Then dashboard is displayed
```

- Each line maps to a **Step Definition** — a Java method annotated `@Given/@When/@Then` that calls your Page Object methods.
- The **Runner** class uses `@CucumberOptions` (features path, glue path, reporters).
- Stack: **Cucumber (Gherkin) + Selenium (browser) + TestNG/JUnit (runner)** — POM still sits underneath.
- **BDD vs TDD (classic):** TDD = developer-first, write failing unit tests before code. BDD = behavior-first, shared language between business, QA and dev at the scenario level.

> **MEMORY HOOK:** *"Given-When-Then = Arrange-Act-Assert in plain English."*

18.9 Appium in one breath

Mobile automation tool that reuses the WebDriver protocol — "Selenium for native/hybrid mobile apps" on Android & iOS (and WinAppDriver for Windows desktop). Same Page Object thinking applies. Say: "Same W3C protocol, different drivers — UiAutomator2 for Android, XCUITest for iOS."

18.10 QA process essentials (hybrid/manual interviews)

- **STLC:** requirement analysis -> test planning -> test case design -> environment setup -> execution -> defect reporting -> closure.
- **Defect lifecycle:** New -> Assigned -> Open/Fixed -> Retest -> Closed — with Rejected, Duplicate, Deferred, Reopen branches.
- **Severity vs priority (THE classic):** severity = HOW BAD the damage is (technical impact); priority = HOW FAST to fix it (business urgency). A misspelled company name on the homepage = LOW severity, HIGH priority.
- **Test design techniques:** equivalence partitioning (test one value per group), boundary value analysis (test at the edges: 0, 1, max, max+1) — interviewers ask you to define both.
- **Agile:** tester's role in sprint planning, daily standup, refinement, review, retrospective — "QA is involved from requirement refinement, not after coding ends."